

滌生大数据之数据同步

修订记录

日期	版本	修改描述	作者	审核
2021-11-26	V1.0	第一稿	健哥	光哥

目 录

1 导学.....	5
1.1 课程概述.....	5
1.2 数据同步是什么☆☆	5
1.3 架构图☆☆	6
1.4 企业中的用法☆	6
2 数据同步方案.....	10
2.1 数据库直连同步☆☆	10
2.2 数据文件同步☆☆	12
2.3 数据库日志解析同步☆☆	14
3 数据同步方式.....	15
3.1 增量同步☆☆	15
3.2 全量同步☆☆	15
3.3 增量合并全量同步☆☆	16
4 数据同步相关技术.....	18
4.1 离线-Sqoop☆☆	18
4.1.1 sqoop 安装.....	19
4.1.2 sqoop 常见的命令及参数☆☆☆	20
4.1.2.1 常见的命令	20
4.1.2.2 公用参数：数据库连接	20
4.1.2.3 公用参数：import	21
4.1.2.3 公用参数：export.....	21
4.1.2.3 公用参数：hive.....	21
4.1.3 sqoop 导入 import.....	22
4.1.3.1 全量导入 mysql 表数据到 HDFS.....	22
4.1.3.2 全量导入 mysql 表数据到 HIVE	23
4.1.3.3 导入表数据子集(where 过滤)	23
4.1.3.4 增量导入	24
4.1.4 sqoop 到处 export.....	26
4.1.5 Sqoop 的缺点.....	27
4.2 离线-DataX☆☆	28
4.2.1 基本原理.....	30
4.2.1.1 架构	30
4.2.1.2 相关概念	30
4.2.1.3 安装	31

4.2.3 使用案例.....	32
4.2.3.1 使用前说明.....	32
4.2.3.1 读取 MySQL 中的数据存放到 HDFS.....	33
4.2.3 DataX 的性能优化☆☆☆	37
4.2.3.1 速度控制	37
4.2.3.2 内存调整	38
4.3 实时-Maxwell☆☆	39
4.3.1 Maxwell 工作原理.....	39
4.3.2 Maxwell 与 Canal 的对比	40
4.3.3 Maxwell 使用.....	40
4.3.3.1 Maxwell 安装	40
4.3.3.2 Maxwell 进程启动.....	41
4.3.4 Maxwell 的缺点.....	42
4.4 实时-Flink CDC☆☆	43
4.4.1 flink 与 flink cdc 的版本对应	44
4.4.2 CDC 实现的原理	44
4.4.3 为什么选择 Flink	45
4.4.4 Flink CDC 特性.....	45
4.4.5 用法实例	46
4.4.5.1 DataStream API 的用法.....	48
4.4.5.2 Flink SQL API 的用法.....	49
4.4.5.3 生产案例: MySQLToMySQL	50
4.5 文件同步-Flume☆☆	52
4.5.1 相关概念.....	52
4.5.2 企业典型案例.....	55
4.5.2.1 模拟端口发送数据	55
4.5.2.2 监控文件的变化写到 HDFS 中	56
4.5.2.3 关于 KafkaChannel	58
4.5.3 企业面试题.....	59
4.6 综合同步解决方案-Seatunnel☆☆	60
4.6.1 特点及使用场景.....	61
4.6.2 架构设计	62
4.6.3 执行引擎类型及支持的数据源	63
4.6.4 安装与使用.....	64
4.6.5 案例 1: 入门项目	66
4.6.5 案例 2: transform Split	68
5 数据同步常见的问题.....	70
5.1 数据源多、可选组件多的问题☆☆	70

5.2 零点漂移的问题☆☆	71
5.2.1 原因分析	71
5.2.2 解决方案	72
5.3 数据倾斜☆☆	73
5.3.1 原因分析	73
5.3.2 数据倾斜现象	74
5.3.3 数据倾斜的危害	74
5.3.3.1 任务长时间挂起，资源利用率下降	74
5.3.3.2 引发内存溢出，导致任务失败	75
5.3.3.3 作业执行时间超出预期，导致后续依赖数据结果的作业出错	75
5.3.4 处理方案	75
5.3.4.1 参数优化	75
5.3.4.2 SQL 优化	76
5.3.4.3 自定义 partitioner (对代码有要求)	82

1 导学

1.1 课程概述

☆表示了解有个印象即可，当别人说起时不至于一脸懵逼

☆☆表示小白必看必会必掌握

☆☆☆表示进阶了解甚至掌握会用，小白可跳过

1.2 数据同步是什么☆☆

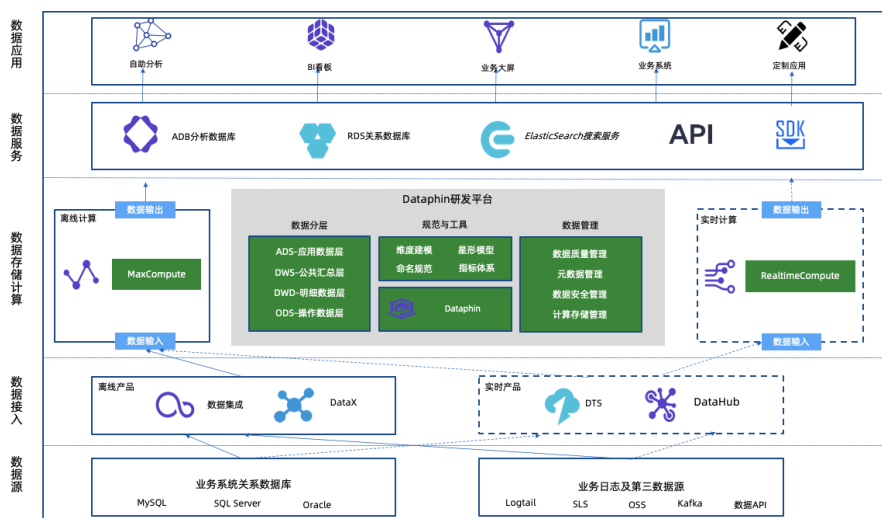
一般，我们说把业务系统数据库如 MySQL、Oracle、MongoDB 的库表数据、日志如埋点等数据以及第三方等其他数据同步至数据仓库之中，称为**数据同步**，**一般同步过来的数据直接原封不动的存放在 ODS (Operational Data Store) 贴源层**。数据接入是数据入仓的第一关，数据接入后一般可以开发一些**数据对账任务**（对账系统就是要及时发现哪些数据表缺失，然后系统或者业务人员根据这些缺失数据表及时进行干预处理，确保同步前后保证数据的一致性。），或者配置一些数据质量监控规则，在数据入仓后就做好第一道数据质量防线。

数据接入后，数据表要做好生命周期管理，防止数据存储成本膨胀增长。数据接入要定好规范，**防止影响到线上服务，比如大批量同步数据的话要避免业务高峰期再执行。**

那么同步至数据仓库前的据源有哪些呢？

- ①关系型数据库的结构化数据：例如 mysql/oracle/sqlserver 等
- ②非关系型数据库的非结构化数据：例如 hbase/mongodb/oceanbase 等
- ③文件系统的结构化和非结构化数据：例如阿里云对象存储 OSS/文件存储 NAS 等
- ④第三方接口、Excel 文件等其他形式的数据源

1.3 架构图☆☆

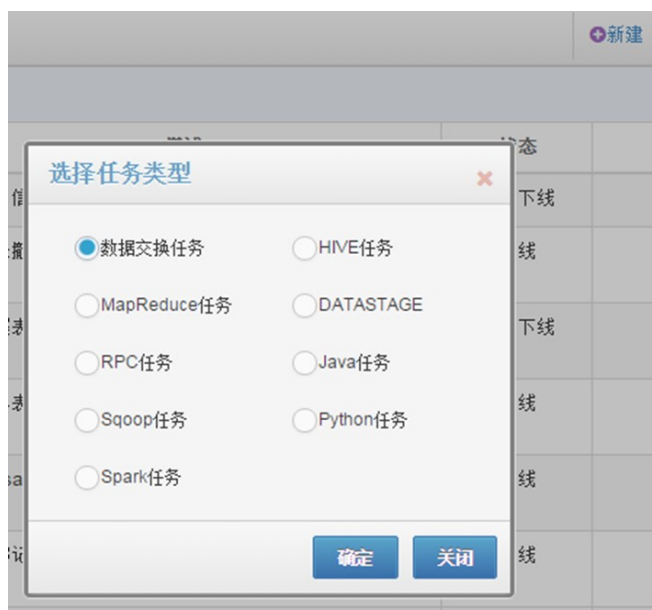


以上是基于阿里云数据中台整体解决方案，对数据中台技术实现进行选型及设计，典型的数据中台技术架构如图所示，**整个技术架构选型包含五个层次：业务数据源存储技术、数据源接入技术、数据中台数据存储与计算技术、数据服务及数据应用技术。**

而该架构也是目前市面上应用比较多的数据流转的五层架构，而在本次课程中我们更加关心的是数据接入层。从上层我们可以看到，因为是阿里系的架构，所有一整套数据接入层用的全是阿里系的产品，包括离线和实时。因为以上很多产品大多是云产品，是付费产品，而我们在课程的讲解过程中主要是以开源产品为主。

1.4 企业中的用法☆

任务管理，点新建选择数据交换任务



涤生大数据

选择源数据源类，例如 MySQL



添加类型，添加数据源



配置基本信息，选择数据源名

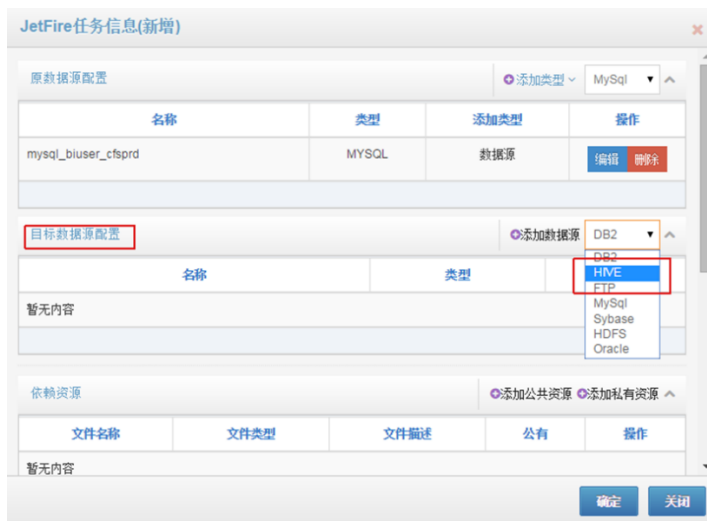


配置导出配置，填写采集 SQL

涤生大数据



选择目标源数据源类，选择 HIVE



添加数据源，配置基本信息，选择数据源名



涤生大数据

配置导入配置，填写目标表和前置 SQL，删除相应分区

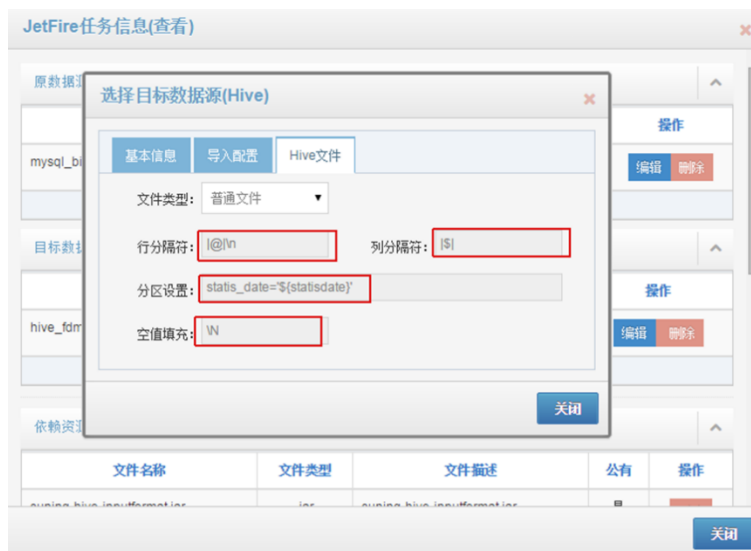


配置 HIVE 文件，填写行分隔符、列分隔符、分区设置和空值填充

行分隔符|@|\n

列分隔符|\$|

空值填充\N



2 数据同步方案

2.1 数据库直连同步☆☆

数据库直连同步是指通过定义好的规范接口 API 和基于动态链接库的方式直接连接业务库，如 ODBC/JDBC 等规定了统一规范的标准接口，不同的数据库基于这套标准接口提供规范的驱动，支持完全相同的函数调用和 SQL 实现。



目前企业中，一般的直连同步均是通过 jdbc 的方式连接业务数据库。

```

object MysqlToHive {
  def main(args: Array[String]): Unit = {
    val spark = SparkSession.builder()
      .master("local")
      .appName("mysql to hive")
      .enableHiveSupport()
      .getOrCreate()

    // 隐式转换
    import spark.implicits._

    // 配置 mysql 数据源
    val df = spark.read
      .format("jdbc")
      .option("url", "jdbc:mysql://ds-bigdata-001:3306/ds_test?useSSL=false")
      .option("driver", "com.mysql.jdbc.Driver")
      .option("user", "ds_read")
      .option("password", "ds#readA0906")
      .option("dbtable", "stud")
      .load()
  }
}
    
```

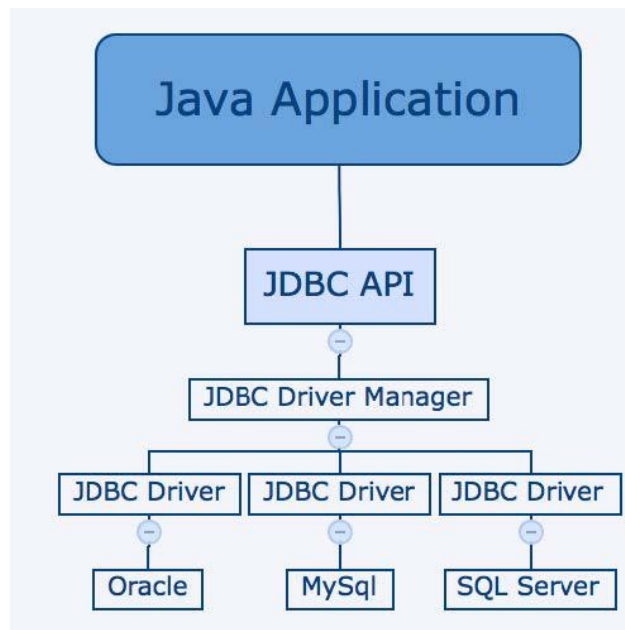
```
// 创建一个临时视图
df.createTempView("stud")

// hive 先创建动态分区表
spark.sql("create table ds_spark.stud1(name string, age int)")

// 动态分区表建好之后就要往里面导入数据
spark.sql("insert overwrite table ds_spark.stud1 select name,age
from stud")

}
```

以上是企业中连接 mysql 的方式，如果你需要连接 Oracle、SQLServer 等数据库，只需要将上述的 jdbc 换成指定的数据库协议即可。如下一图谨记：



直连同步这也是目前大数据平台离线同步数据的主要手段，但**该方式对业务源系统的性能影响可能会较大（因为读取数据表的同时会影响业务库的 qps）**，比如涉及到大批量数据同步时会拖垮业务系统的性能，如果业务数据库有采用主从库策略的话，那么我们从数据库抽取数据还可以避免对业务系统的影响，但是对于数据量大的表同步，效率将不会太高。

数据库直连同步的优缺点：

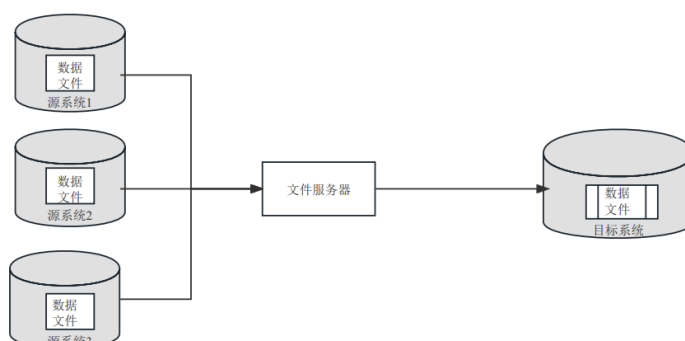
优点：实现简单，是大数据平台离线同步的主要手段。

缺点：同步数据量大的表，会影响业务系统的性能，谨慎操作，**数据库若有主从库策**

略则要从从库抽数避免对业务系统直接影响，而且还要制定数据接入规范，避免业务高峰期大批量抽数操作，不然随时会带来影响线上服务的风险。

2.2 数据文件同步☆☆

互联网是不安全的，很多涉密的数据是没有办法直接从公网进行直接拉的，这个时候数据就要以数据文件的形式进行线上或者线下传输。数据文件同步通过约定好的文件编码、大小、格式等，直接从源系统生成数据的文本文件，由专门的文件服务器，如 FTP 服务器传输到目标系统后，加载到目标数据库系统中；甚至涉密级别更高的一些数据直接走线下的 U 盘、硬盘进行来回拷贝。



互联网的日志类数据通常是以文件形式保存的，适合这种同步方式。不过如果通过文件服务器上传下载的话难免会有丢包或者错误的风险，所以在传输数据文件的同时，为了确保数据文件同步的完整性，会一并传输一个校验文件，该校验文件记录了数据文件的数据量以及文件大小等校验信息，以供下游目标系统验证数据同步的准确性。目前在大数据的下载平台上支持三种文件校验算法：**asc、md5 及 sha1**，如下：

org/apache/spark/spark-core_2.12/3.3.0

../	2022-06-09 23:03	4307158
spark-core_2.12-3.3.0-javadoc.jar	2022-06-09 23:03	862
spark-core_2.12-3.3.0-javadoc.jar.asc	2022-06-09 23:03	33
spark-core_2.12-3.3.0-javadoc.jar.md5	2022-06-09 23:03	41
spark-core_2.12-3.3.0-javadoc.jar.sha1	2022-06-09 23:03	2430727
spark-core_2.12-3.3.0-sources.jar	2022-06-09 23:03	862
spark-core_2.12-3.3.0-sources.jar.asc	2022-06-09 23:03	33
spark-core_2.12-3.3.0-sources.jar.md5	2022-06-09 23:03	41
spark-core_2.12-3.3.0-sources.jar.sha1	2022-06-09 23:03	1291852
spark-core_2.12-3.3.0-test-sources.jar	2022-06-09 23:03	862
spark-core_2.12-3.3.0-test-sources.jar.asc	2022-06-09 23:03	33
spark-core_2.12-3.3.0-test-sources.jar.md5	2022-06-09 23:03	41
spark-core_2.12-3.3.0-test-sources.jar.sha1	2022-06-09 23:03	5250367
spark-core_2.12-3.3.0-tests.jar	2022-06-09 23:03	862
spark-core_2.12-3.3.0-tests.jar.asc	2022-06-09 23:03	33
spark-core_2.12-3.3.0-tests.jar.md5	2022-06-09 23:03	41
spark-core_2.12-3.3.0-tests.jar.sha1	2022-06-09 23:03	10997851
spark-core_2.12-3.3.0.jar	2022-06-09 23:03	862
spark-core_2.12-3.3.0.jar.asc	2022-06-09 23:03	33
spark-core_2.12-3.3.0.jar.md5	2022-06-09 23:03	41
spark-core_2.12-3.3.0.jar.sha1	2022-06-09 23:03	37778
spark-core_2.12-3.3.0.pom	2022-06-09 23:03	862
spark-core_2.12-3.3.0.pom.asc	2022-06-09 23:03	33
spark-core_2.12-3.3.0.pom.md5	2022-06-09 23:03	41
spark-core_2.12-3.3.0.pom.sha1	2022-06-09 23:03	

另外还可以在源系统生成数据文件的同时进行压缩和加密操作，在目标系统接收数据

文件时再进行解压缩和解密操作，保证数据文件同步的效率和安全性。

数据文件同步的优缺点：

优点：当数据源包含多个异构的数据库系统时，用这种方式比较简单、实用。另外，日志类数据通常是以文本文件形式存在的，也适合使用数据文件同步方式。

缺点：通过服务器上传和下载的话难免会出现丢包或者错误的风险，保证数据文件同步的完整性和安全性以及传输效率，可在传输数据文件的同时需要一并传输一个校验文件供目标系统校验，同时在传输存储较大文件时需要对数据文件进行加密和压缩操作

```
object CsvSource {
  def main(args: Array[String]): Unit = {
    val spark = SparkSession.builder()
      .master("local")
      .appName("csv")
      .getOrCreate()

    // 隐式转换
    import spark.implicits._

    // 使用 header 选项
    // 使用 schema 参数
    val schema =
      """
        |name string,
        |age int,
        |height int
        |""".stripMargin

    val df = spark.read
      // .option("header",true)
      .schema(schema)
      .csv("src/main/resources/data/people.csv")

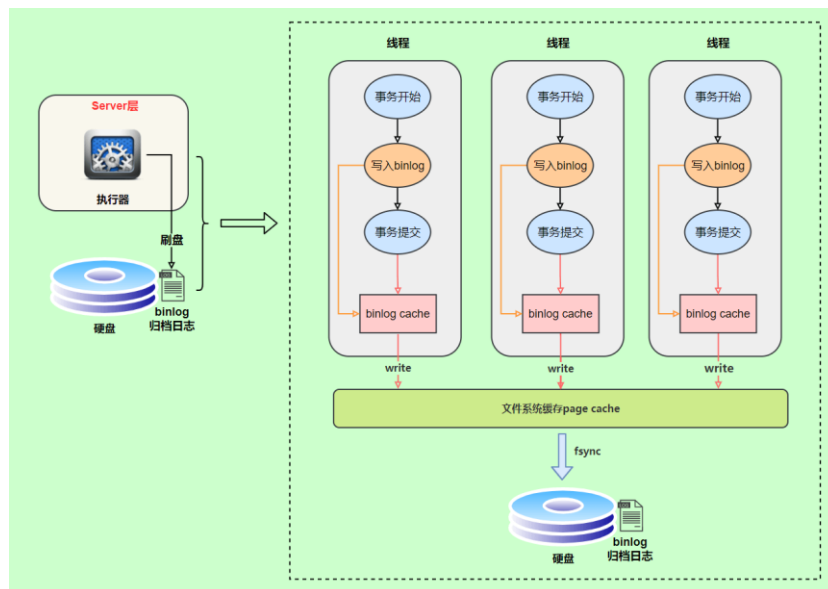
    df.write.csv("src/main/resources/data/people1.csv")
  }
}
```



2.3 数据库日志解析同步☆☆

大多数主流数据库都已经实现了使用**日志文件 (binlog)** 进行系统恢复，因为日志文件信息足够丰富，而且数据格式也很稳定，完全可以通过解析日志文件获取发生变更的数据 (**CDC: Change Database Capture**)。因此有了这一机制，我们可以通过监控该文件，通过该文件的变动从而满足全量或者增量数据同步的需求，比如 mysql，一般是通过解析 binlog 日志方式来获取增量的数据更新，并通过消息订阅模式来实现数据的**实时同步** 3002

Mysql 的 binlog 一般的产生如下：



数据库日志解析同步的优缺点：

优点：

1. 日志文件信息足够丰富，数据格式稳定，通过解析日志文件满足增量数据同步的需求。
2. 通过网络协议，确保数据文件的正确接收，提供网络传输冗余，保证数据的完整性。
3. 实现了实时与准实时同步的能力，延迟控制在毫秒级别。

缺点：

1. 当数据更新量超出系统处理峰值，会导致数据延迟。
2. 投入较大，需要在源数据库和目标数据库之前部署一个实时数据同步系统。
3. 数据漂移和遗漏。

3 数据同步方式

为保证统计结果的正确性，需要保证数据仓库中的数据与业务数据库是同步的，离线数仓的计算周期通常为天，所以数据同步周期也通常为天，即每天同步一次即可（即所谓的 T+1 同步）。数据的同步策略有全量同步和增量同步。

3.1 增量同步☆☆

增量同步，就是每天只将业务数据中的新增及变化数据同步到数据仓库。采用每日增量同步的表，通常需要在首日先进行一次全量同步。如下：

```
insert overwrite table ods.table_in partition (dt='${yyyy-MM-dd}') select *
from table_source -- 业务源表
where create_time >= '调度当日时间'
or update_time >= '调度当日时间'
```

这样每日增量同步过来的数据都放在**增量分区表的每日分区内**，可以保存数据的历史变更状态，不过要取历史全量数据的数仓最新状态数据对于增量分区表的话就会取数麻烦些。比如要取全量订单记录当前最新状态，需要先扫描增量分区表历史全部分区得到全量订单数据，再取每笔订单数仓最新的那条状态记录。

备注：离线增量和实时增量的区 522B

- 1、离线增量一般是做 T+1 的处理方式，即次日会同步前一日的所有的所需要的业务数据
- 2、实时增量一般是实时监测，只要所需要的业务数据发生变化，就将该数据立马同步到数仓之中

3.2 全量同步☆☆

全量同步，就是每天都将业务数据库中的全部数据同步一份到数据仓库，这是保证两侧数据同步的最简单的方式。

对于全量表，一般是有两种，一种是普通的全量表，如下

```
insert overwrite table ods.table_tm
select *
from table_source -- 业务源表
```

这样的全量表在实际的生产之中一般很少使用，因为每次往全量表里面写数据的时候都会覆盖之前的全量表。

另外一种就是**分区全量表（又称为快照表）**，下游要取历史全量数据最新状态数据就很便捷了，只需要取全量分区表的最新分区的全量数据即可，就代表着全量数据且数仓的最新状态。

```
insert overwrite table ods.table_tm partition (dt='${yyyy-MM-dd}')
select *
from table_source -- 业务源表
```

不过全量同步这种抽取数据的方式不建议使用，数据量小还行，因为每日分区存储一份历史全量数据会对 HDFS 存储造成浪费是一方面，当数据量大的话这样做是绝不可行的，会拖垮业务源系统数据库，对业务源数据库造成压力负载，严重时还会造成线上故障。所以建议一般对于小数据量可以选择全量同步，而对于大数据量的数据则选择增量同步

3.3 增量合并全量同步☆☆

基于以上增量同步和全量同步 2 种同步数据方式的利弊，**结合起来形成一种同步 T+1 的增量数据，再合并历史 T+2 的全量数据的抽取数据的方式**，这样就**在最新分区表里即满足了数仓的全量最新快照数据，同时也降低了对存储的浪费以及对业务源数据库的压力负载，兼容了上述 2 种策略**。以下附 sql 技术代码案例讲解：

```
-- 增量合并全量方案一：union all
insert overwrite table 全量表 partition (dt = '${yyyy-MM-dd}')
select *
from
(
    select
        *,row_number() over (partition by 去重主键 order by 时间字段
desc) as rk
    from
    (
        select *
        from 增量分区表
        where 1 = 1
        and dt = '${yyyy-MM-dd}' -- 增量表 T+1 当前增量数据
    union all
        select *
        from 全量分区表
```



```

        where 1 = 1
        and dt = '${yyyy-MM-dd,-1d}' -- 全量表 T+2 历史全量数据
    ) a
) b
where rk = 1; -- 取最新状态数据

-- 增量合并全量方案二：full outer join
insert overwrite table 全量表 partition (dt = '${yyyy-MM-dd}')
select
    -- 优先取增量表最新数据更新覆盖
    coalesce(a.主键,b.主键) as 主键
    ,...
from
(
    select *
    from 增量分区表
    where dt = '${yyyy-MM-dd}' -- 增量表 T+1 当前增量数据
) a
full outer join
(
    select *
    from 全量分区表
    where dt = '${yyyy-MM-dd,-1d}' -- 全量表 T+2 历史全量数据
) b
on a.主键 = b.主键
    
```

关于增量合并全量方案的**小结**：

方案一和方案二这 2 种解决方案的**本质上都是用增量数据来更新覆盖历史全量数据**，不过通过 full outer join 方式合并前要确保记录是唯一的，这样关联时才不会发生数据发散。

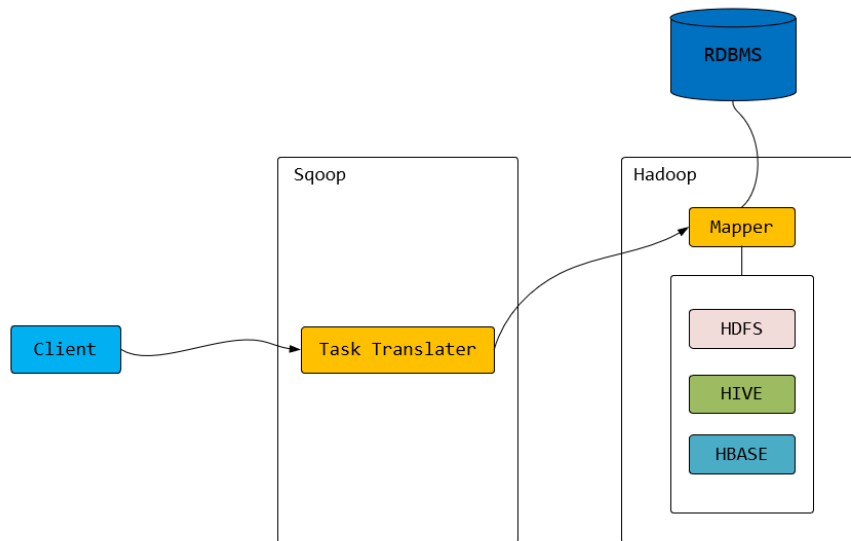
若说 2 种解决方案的性能对比的话就大差不差吧，**数据量大的话方案一可能发生的性能瓶颈主要在合并过程的全局排序去重**，**方案二可能发生的性能瓶颈主要在合并过程的大表关联上**，目前在行业中离线数据接入大致也是这种思路-增量合并全量。

4 数据同步相关技术

4.1 离线-Sqoop☆☆

Apache Sqoop (SQL-to-Hadoop) 是在 Hadoop 生态体系和 RDBMS 体系之间传送数据的一种工具。来自于 Apache 软件基金会提供。

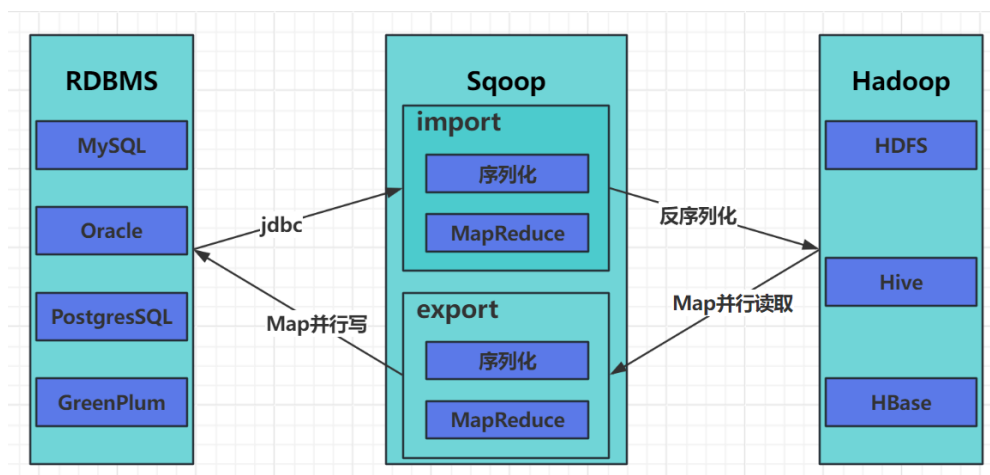
Sqoop 工作机制是将导入或导出命令翻译成 MapReduce 程序来实现。在翻译出的 MapReduce 中主要是对 inputformat 和 outputformat 进行定制。整体逻辑如下：



Hadoop 生态系统包括：HDFS、Hive、Hbase 等

RDBMS 体系包括：Mysql、Oracle、DB2 等

Sqoop 可以理解为：“SQL 到 Hadoop 和 Hadoop 到 SQL 的工具”。因此



Sqoop 主体上主要就涉及到两个命令：import 和 export，从上图中可以看出：

Import: 数据导入。RDBMS----->Hadoop

Export: 数据导出。Hadoop----->RDBMS

另外从上图同时也可以看出，Sqoop 执行的整个过程只有 map 阶段，没有 reduce 阶段（考点）

4.1.1 sqoop 安装

安装 sqoop 的前提是已经具备 java 和 hadoop 的环境。

配置文件修改：

```
cd $SQOOP_HOME/conf
mv sqoop-env-template.sh sqoop-env.sh
vi sqoop-env.sh

    export HADOOP_COMMON_HOME= /export/servers/hadoop-2.7.5
    export HADOOP_MAPRED_HOME= /export/servers/hadoop-2.7.5
    export HIVE_HOME= /export/servers/hive
```

加入 mysql 的 jdbc 驱动包

```
cp /hive/lib/mysql-connector-java-5.1.32.jar $SQOOP_HOME/lib/
```

备注说明：在 ds 大数据平台，已经为同学们构建了一个全局的 sqoop 命令，所以同学们在使用的時候直接执行 sqoop 的命令即可。

```
[whujian888@ds-bigdata-005 jars]$ sqoop help
Warning: /opt/cloudera/parcels/CDH-6.2.1-1.cdh6.2.1.p0.1580995/bin/../lib/sqoop/./accumulo does not exist! Accumulo imports will fail.
Please set $ACCUMULO_HOME to the root of your Accumulo installation.
SLF4J: Class path contains multiple SLF4J bindings.
SLF4J: Found binding in [jar:file:/opt/cloudera/parcels/CDH-6.2.1-1.cdh6.2.1.p0.1580995/jars/slf4j-log4j12-1.7.25.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: Found binding in [jar:file:/opt/cloudera/parcels/CDH-6.2.1-1.cdh6.2.1.p0.1580995/jars/log4j-slf4j-impl-2.0.2.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: See http://www.slf4j.org/codes.html#multiple_bindings for an explanation.
SLF4J: Actual binding is of type [org.slf4j.impl.Log4jLoggerFactory]
23/11/18 17:22:03 INFO sqoop.Sqoop: Running Sqoop version: 1.4.7-cdh6.2.1
usage: sqoop COMMAND [ARGS]

Available commands:
codegen      Generate code to interact with database records
create-hive-table  Import a table definition into Hive
eval         Evaluate a SQL statement and display the results
export       Export an HDFS directory to a database table
help         List available commands
import       Import a table from a database to HDFS
import-all-tables  Import tables from a database to HDFS
import-mainframe  Import datasets from a mainframe server to HDFS
job           Work with saved jobs
list-databases  List available databases on a server
list-tables   List available tables in a database
merge        Merge results of incremental imports
metastore    Run a standalone Sqoop metastore
version      Display version information

See 'sqoop help COMMAND' for information on a specific command.
```

验证启动：

```
#本命令会列出所有 mysql 的数据库。
sqoop list-databases \
--connect jdbc:mysql://ds-bigdata-001:3306/ \
--username ds_read --password ds#readA0906
#本命令会列出所有 mysql 的数据库。
sqoop list-tables \
--connect jdbc:mysql://ds-bigdata-001:3306/ds_test \
--username ds_read \
```

```
--password ds#readA0906 \
```

4.1.2 sqoop 常见的命令及参数☆☆☆

4.1.2.1 常见的命令

命令	类	说明
import	ImportTool	将数据导入到集群
export	ExportTool	将集群数据导出
codegen	CodeGenTool	获取数据库中某张表数据生成Java并打包Jar
create-hive-table	CreateHiveTableTool	创建Hive表
eval	EvalSqlTool	查看SQL执行结果
import-all-tables	ImportAllTablesTool	导入某个数据库下所有表到HDFS中
job	JobTool	用来生成一个sqoop的任务，生成后，该任务并不执行，除非使用命令执行该任务。
list-databases	ListDatabasesTool	列出所有数据库名
list-tables	ListTablesTool	列出某个数据库下所有表
merge	MergeTool	将HDFS中不同目录下面的数据合在一起，并存放在指定的目录中
metastore	MetastoreTool	记录sqoop job的元数据信息，如果不启动metastore实例，则默认的元数据存储目录为：~/sqoop，如果要更改存储目录，可以在配置文件sqoop-site.xml中进行更改。
help	HelpTool	打印sqoop帮助信息
version	VersionTool	打印sqoop版本信息

4.1.2.2 公用参数：数据库连接

参数	说明
-connect	连接关系型数据库的URL
-connection-manager	指定要使用的连接管理类
-driver	Hadoop根目录
-help	打印帮助信息
-password	连接数据库的密码
-username	连接数据库的用户名
-verbose	在控制台打印出详细信息

4.1.2.3 公用参数：import

参数	说明
-enclosed-by	给字段值前加上指定的字符
-escaped-by	对字段中的双引号加转义符
-fields-terminated-by	设定每个字段是以什么符号作为结束，默认为逗号
-lines-terminated-by	设定每行记录之间的分隔符，默认是\n
-mysql-delimiters	Mysql默认的分隔符设置，字段之间以逗号分隔，行之间以\n分隔，默认转义符是\，字段值以单引号包裹。
-optionally-enclosed-by	给带有双引号或单引号的字段值前后加上指定字符。

4.1.2.3 公用参数：export

参数	说明
-input-enclosed-by	对字段值前后加上指定字符
-input-escaped-by	对含有转移符的字段做转义处理
-input-fields-terminated-by	字段之间的分隔符
-input-lines-terminated-by	行之间的分隔符
-mysql-delimiters	Mysql默认的分隔符设置，字段之间以逗号分隔，行之间以\n分隔，默认转义符是\，字段值以单引号包裹。

4.1.2.3 公用参数：hive

参数	说明
-hive-delims-replacement	用自定义的字符串替换掉数据中的\r\n和\013 \010等字符
-hive-drop-import-delims	在导入数据到hive时，去掉数据中的\r\n\013\010这样的字符
-map-column-hive	生成hive表时，可以更改生成字段的数据类型
-hive-partition-key	创建分区，后面直接跟分区名，分区字段的默认类型为string
-hive-partition-value	导入数据时，指定某个分区的值
-hive-home	hive的安装目录，可以通过该参数覆盖之前默认配置的目录
-hive-import	将数据从关系数据库中导入到hive表中
-hive-overwrite	覆盖掉在hive表中已经存在的数据
-create-hive-table	默认是false，即，如果目标表已经存在了，那么创建任务失败。
-hive-table	后面接要创建的hive表,默认使用MySQL的表名
-table	指定关系数据库的表名

4.1.3 sqoop 导入 import

从 RDBMS 中以 MySQL 为例导出单表到 HDFS, **MySQL 表中的每一行数据被视为 HDFS 中的一条记录, 所有记录都存储为文本数据。**下面的语法格式将用于将数据导入 HDFS。

```
sqoop import (generic-args) (import-args)
```

4.1.3.1 全量导入 mysql 表数据到 HDFS

下面的命令用于从 MySQL 数据库服务器中的 emp 表导入 HDFS。

```
sqoop import \  
--connect jdbc:mysql://ds-bigdata-001:3306/ds_test \  
--username ds_read \  
--password ds#readA0906 \  
--delete-target-dir \  
--target-dir /test_demo/sqoop_test/ \  
--table stud \  
--m 1
```

其中:

- delete-target-dir**: 如果路径存在就自动删除
- target-dir**: 可以用来指定导出数据存放至 HDFS 的目录;
- m**: 是--num-mappers 的缩写, 表示启动 N 个 mapper 任务并行, 默认 4

为了验证在 HDFS 导入的数据, 在导入完成之后可以使用命令查看

```
[whujian888@ds-bigdata-005 ~]$ hdfs dfs -ls /test_demo/sqoop_test/  
Found 2 items  
-rw-r--r--  2 whujian888 supergroup      0 2023-11-18 20:09 /test_demo/sqoop_test/_SUCCESS  
-rw-r--r--  2 whujian888 supergroup    25 2023-11-18 20:09 /test_demo/sqoop_test/part-m-00000  
[whujian888@ds-bigdata-005 ~]$ hdfs dfs -cat /test_demo/sqoop_test/part-m-00000  
Tom,18  
Alice,17  
Nancy,19
```

可以看出它会在 HDFS 上默认用逗号,分隔 emp 表的数据和字段。可以通过:

--fields-terminated-by '\t': 来指定分隔符。

```
[whujian888@ds-bigdata-005 ~]$ hdfs dfs -ls /test_demo/sqoop_test/  
Found 2 items  
-rw-r--r--  2 whujian888 supergroup      0 2023-11-18 20:12 /test_demo/sqoop_test/_SUCCESS  
-rw-r--r--  2 whujian888 supergroup    25 2023-11-18 20:12 /test_demo/sqoop_test/part-m-00000  
[whujian888@ds-bigdata-005 ~]$ hdfs dfs -cat /test_demo/sqoop_test/part-m-00000  
Tom      18  
Alice    17  
Nancy    19
```

4.1.3.2 全量导入 mysql 表数据到 HIVE

先将关系型数据的表结构复制到 hive

```
sqoop create-hive-table \  
--connect jdbc:mysql://ds-bigdata-001:3306/ds_test \  
--username ds_read \  
--password ds#readA0906 \  
--table stud \  
--hive-table stud
```

备注：默认是 false，即，如果目标表已经存在了，那么创建任务失败。

从关系数据库导入文件到 hive 中

```
sqoop import \  
--connect jdbc:mysql://ds-bigdata-001:3306/ds_test \  
--username ds_read \  
--password ds#readA0906 \  
--table stud \  
--hive-table stud \  
--hive-import \  
--m 1
```

4.1.3.3 导入表数据子集(where 过滤)

where 可以指定从关系数据库导入数据时的查询条件。它执行在数据库服务器相应的 SQL 查询，并将结果存储在 HDFS 的目标目录。

```
sqoop import \  
--connect jdbc:mysql://ds-bigdata-001:3306/ds_test \  
--username ds_read \  
--password ds#readA0906 \  
--table stud \  
--where "name = ' Tom' " \  
--hive-table stud \  
--hive-import \  
--m 1
```

备注：默认是是 append 的模式，可以指定--hive-overwrite 进行数据覆

4.1.3.4 增量导入

在实际工作当中，数据的导入，很多时候都是只需要导入增量数据即可，并不需要表中的数据每次都全部导入到 hive 或者 hdfs 当中去，这样会造成数据重复的问题。因此一般都是选用一些字段进行增量的导入，**sqoop 支持增量的导入数据，用于只导入比已经导入行新的数据行。**其中，使用的参数为：

1、--check-column col

用来指定一些列，这些列在增量导入时**用来检查这些数据是否作为增量数据进行导入**，和关系型数据库中的自增字段及时间戳类似。

注意：这些被指定的列的类型不能使任意字符类型，如 char、varchar 等类型都是不可以的，同时-- check-column 可以去指定多个列。

2、--incremental mode

增量导入数据分为两种方式：

- i、append：基于递增列的增量数据导入，必须为数值型
- li、lastmodified：基于时间列的数据增量导入，必须为时间戳类型

3、--last-value value

指定自从上次导入后列的最大值（大于该指定的值），也可以自己设定某一值

案例 1：append 模

Mysql 中有表 stu_inc：

```
DROP TABLE IF EXISTS `stu_inc`;
CREATE TABLE `stu_inc` (
  `id` int(11) NOT NULL AUTO_INCREMENT,
  `name` varchar(100) DEFAULT NULL,
  `school` varchar(100) DEFAULT NULL,
  PRIMARY KEY (`id`)
) ENGINE=InnoDB AUTO_INCREMENT=14 DEFAULT
CHARSET=utf8mb4;

SET FOREIGN_KEY_CHECKS = 1;
```

插入几条数据：

```
insert into stu_inc(id, name, school) VALUES
```



```
(1,'Tom','Qinghua'),
(2,'Tom1','Qinghua1'),
(3,'Tom2','Qinghua2'),
(4,'Tom3','Qinghua3'),
(5,'Tom4','Qinghua4'),
(6,'Tom5','Qinghua5');
```

执行如下命令：

```
sqoop import \
--connect jdbc:mysql://ds-bigdata-001:3306/ds_test \
--username ds_read \
--password ds#readA0906 \
--table stu_inc \
--m 1 \
--target-dir /test_demo/sqoop_test1 \
--incremental append \
--check-column id \
--last-value 3 // 同步时不包含词条数据
```

```
[whujian888@ds-bigdata-005 ~]$ hdfs dfs -ls /test_demo/sqoop_test1/
Found 1 items
-rw-r--r--  2 whujian888 whujian888      48 2023-11-18 20:43 /test_demo/sqoop_test1/part-m-00000
[whujian888@ds-bigdata-005 ~]$ hdfs dfs -cat /test_demo/sqoop_test1/part-m-00000
4,Tom3,Qinghua3
5,Tom4,Qinghua4
6,Tom5,Qinghua5
```

备注： --append and --delete-target-dir can not be used together.

案例 2：lastmodified 模式

Mysql 中有表 stu_last:

```
CREATE TABLE stu_last (
  id INT,
  name VARCHAR ( 100 ),
  last_modified TIMESTAMP DEFAULT CURRENT_TIMESTAMP
ON UPDATE CURRENT_TIMESTAMP
);
```

插入几条数据：

```
insert into stu_last(id, name,last_modified) VALUES
(1,'Tom',timestamp '2023-11-18 20:50:02'),
(2,'Tom1',timestamp '2023-11-18 20:51:02'),
(3,'Tom2',timestamp '2023-11-18 20:52:02'),
(4,'Tom3',timestamp '2023-11-18 20:53:02'),
```

```
(5,'Tom4',timestamp '2023-11-18 20:54:02'),
(6,'Tom5',timestamp '2023-11-18 20:55:02');
```

执行如下命令：

```
sqoop import \
--connect jdbc:mysql://ds-bigdata-001:3306/ds_test \
--username ds_read \
--password ds#readA0906 \
--table stu_last \
--target-dir /test_demo/sqoop_test2 \
--m 1 \
--check-column last_modified \
--incremental lastmodified \
--last-value "2023-11-18 20:52:02" \ // 同步时包含此条数据
```

```
[whujian888@ds-bigdata-005 ~]$ hdfs dfs -ls /test_demo/sqoop_test2/
Found 2 items
-rw-r--r--  2 whujian888 whujian888      0 2023-11-18 21:02 /test_demo/sqoop_test2/_SUCCESS
-rw-r--r--  2 whujian888 whujian888    116 2023-11-18 21:02 /test_demo/sqoop_test2/part-m-00000
[whujian888@ds-bigdata-005 ~]$ hdfs dfs -cat /test_demo/sqoop_test2/part-m-00000
3,Tom2,2023-11-18 20:52:02.0
4,Tom3,2023-11-18 20:53:02.0
5,Tom4,2023-11-18 20:54:02.0
6,Tom5,2023-11-18 20:55:02.0
```

4.1.4 sqoop 到处 export

默认情况下，**sqoop export** 将每行输入记录转换成一条 **INSERT** 语句，添加到目标数据库表中。如果数据库中的表具有约束条件（例如，其值必须唯一的主键列）并且已有数据存在，则必须注意避免插入违反这些约束条件的记录。**如果 INSERT 语句失败，导出过程将失败。**此模式主要用于将记录导出到可以接收这些结果的空表中。**通常用于全表数据导出。**

导出时可以是将 Hive 表中的全部记录或者 HDFS 数据（可以是全部字段也可以部分字段）导出到 Mysql 目标表。

准备 HDFS 数

在 HDFS 文件系统中 “/test_demo/” 目录的下创建一个文件 emp_data.txt：

```
1201,gopal,manager,50000,TP
1202,manisha,preader,50000,TP
1203,kalil,php dev,30000,AC
1204,prasanth,php dev,30000,AC
```

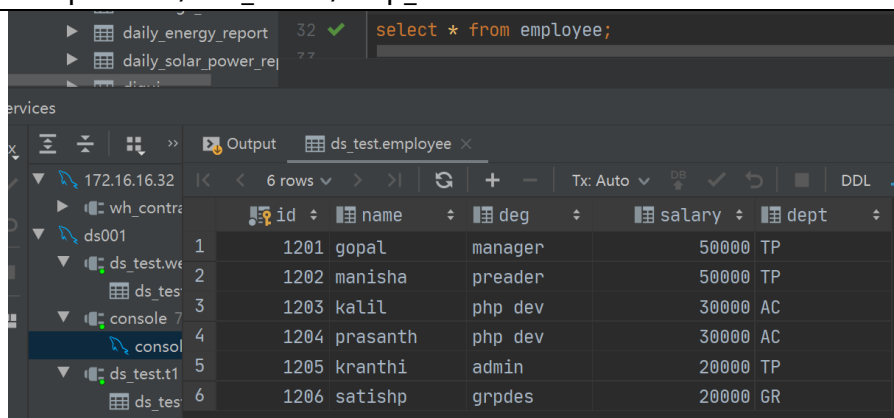
```
1205,kranthi,admin,20000,TP
1206,satishp,grpdes,20000,GR
```

手动创建 mysql 中的目标表

```
CREATE TABLE employee (
  id INT NOT NULL PRIMARY KEY,
  name VARCHAR(20),
  deg VARCHAR(20),
  salary INT,
  dept VARCHAR(10)
);
```

执行导出命令

```
sqoop export \
--connect jdbc:mysql://ds-bigdata-001:3306/ds_test \
--username ds_read \
--password ds#readA0906 \
--table employee \
--export-dir /test_demo/emp_data.txt
```



The screenshot shows a database client interface with a query window at the top containing the command `select * from employee;`. Below the query window, a table titled 'ds_test.employee' displays the results of the query. The table has 6 rows and 6 columns: id, name, deg, salary, and dept. The data is as follows:

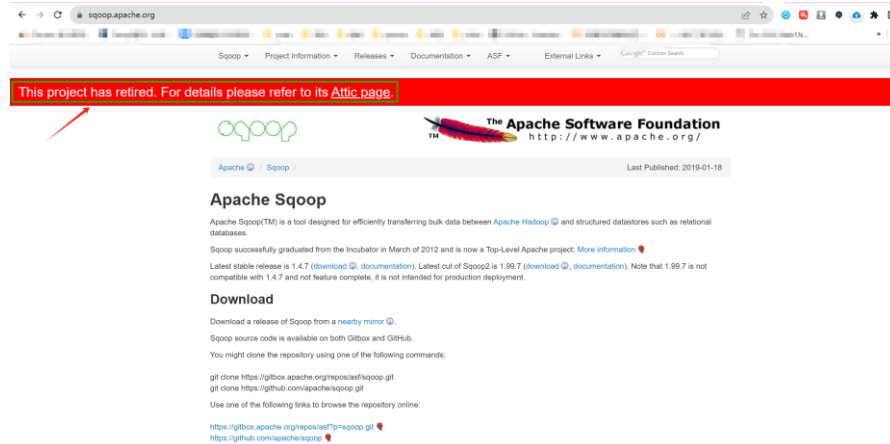
	id	name	deg	salary	dept
1	1201	gopal	manager	50000	TP
2	1202	manisha	preader	50000	TP
3	1203	kalil	php dev	30000	AC
4	1204	prasanth	php dev	30000	AC
5	1205	kranthi	admin	20000	TP
6	1206	satishp	grpdes	20000	GR

4.1.5 Sqoop 的缺点

1、Sqoop 属于 Hadoop 生态圈中一员，和 Hadoop 深度的绑定，深受青睐。但随着新的基于内存计算的批处理框架 Spark 的诞生，传统的 MapReduce 逐渐的淡出人们的视野，而 **Sqoop 底层与 MapReduce 又是强耦合**，不是解耦的。因此，随着企业逐渐 Spark 化，不能与 Spark 结合的 Sqoop 也是被历史淘汰的最大原因之一。

2、Apache Sqoop 在更新最后一个版本 1.4.7 之后就**永久从 Apache 退役了**，因此

如果在使用的过程中如果遇到什么非常棘手的问题，很大可能找不到相应的解决办法，只能通过自己去研读源码，自己去解决相应的问题。这在快速便捷式开发的今天，可能也是被弃用的最大原因之一。

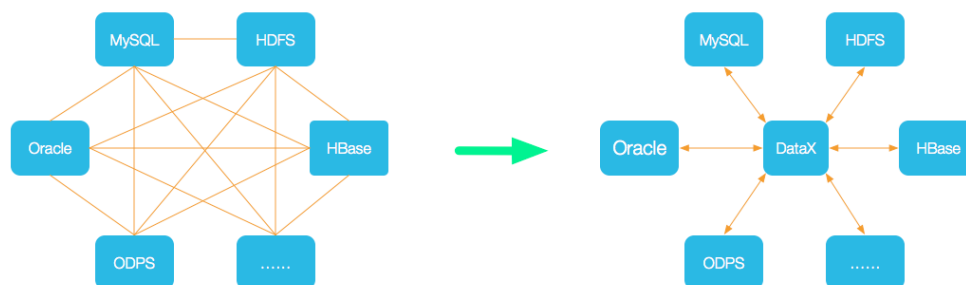


3、只支持常见的 RDBMS 如 MySQL、Oracle 等和常见的 Hadoop 生态圈的 HDFS、Hive 等，对于目前市面上追捧火热的数据分析工具如 StarRocks、Doris、ClickHouse 等的支持为零，主要还是停更的原因。

4.2 离线-DataX☆☆

DataX 是阿里云 DataWorks 数据集成的开源版本，在阿里巴巴集团内被广泛使用的离线数据同步工具/平台，它是一个**异构数据源离线同步工具**，致力于实现包括关系型数据库(MySQL、Oracle 等)、HDFS、Hive、ODPS、HBase、FTP 等各种异构数据源**之间稳定高效的数据同步功能**。

为了解决异构数据源同步问题，**DataX 将复杂的网状的同步链路变成了星型数据链路**，DataX 作为中间传输载体负责连接各种数据源。



当需要接入一个新的数据源的时候，只需要将此数据源对接到 DataX，便能跟已有的数据源做到无缝数据同步。DataX 目前已经有了比较全面的插件体系，主流的 RDBMS 数据库、NOSQL、大数据计算系统都已经接入。DataX 支持目前市面上几乎所有的数据库类型，也是让他在国内大展头角的原因，如下图所示：

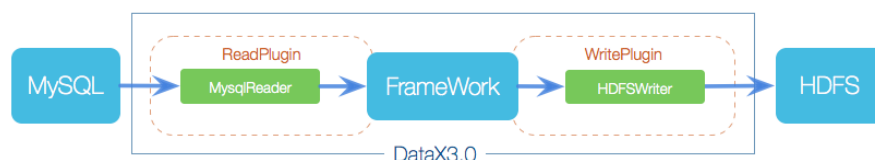
涤生大数据

类型	数据源	Reader(读)	Writer(写)	文档
RDBMS 关系型数据库	MySQL	√	√	读 、 写
	Oracle	√	√	读 、 写
	OceanBase	√	√	读 、 写
	SQLServer	√	√	读 、 写
	PostgreSQL	√	√	读 、 写
	DRDS	√	√	读 、 写
	Kingbase	√	√	读 、 写
	通用RDBMS(支持所有关系型数据库)	√	√	读 、 写
阿里云数仓数据存储	ODPS	√	√	读 、 写
	ADB		√	写
	ADS		√	写
	OSS	√	√	读 、 写
	OCS		√	写
	Hologres		√	写
	AnalyticDB For PostgreSQL		√	写
阿里云中间件	datahub	√	√	读 、 写
	SLS	√	√	读 、 写
图数据库	阿里云 GDB	√	√	读 、 写
	Neo4j		√	写
NoSQL数据存储	OTS	√	√	读 、 写
	Hbase0.94	√	√	读 、 写
	Hbase1.1	√	√	读 、 写
	Phoenix4.x	√	√	读 、 写
	Phoenix5.x	√	√	读 、 写
	MongoDB	√	√	读 、 写
	Cassandra	√	√	读 、 写
数仓数据存储	StarRocks	√	√	读 、 写
	ApacheDoris		√	写
	ClickHouse	√	√	读 、 写
	Databend		√	写
	Hive	√	√	读 、 写
	kudu		√	写
	selectdb		√	写
无结构化数据存储	TxtFile	√	√	读 、 写
	FTP	√	√	读 、 写
	HDFS	√	√	读 、 写
	Elasticsearch		√	写
时间序列数据库	OpenTSDB	√		读
	TSDB	√	√	读 、 写
	TDengine	√	√	读 、 写

4.2.1 基本原理

4.2.1.1 架构

DataX 在阿里巴巴集团内被广泛使用，承担了所有大数据的离线同步业务，并已持续稳定运行了 6 年之久。目前每天完成同步 8w 多道作业，每日传输数据量超过 300TB。而阿里云最新开源全新版本 DataX3.0，有了更多更强大的功能和更好的使用体验。



作为离线数据同步框架，采用 Framework + plugin 架构。将数据源读取、写入抽象成为 Reader/Writer 插件，纳入到整个同步框架：

Reader：数据采集模块，采集数据源的数据，将数据发送给 Framework

Writer：数据写入模块，不断向 Framework 取数据，并将数据写入到目的端

Framework：连接 reader 和 writer，作为两者的数据传输通道，并处理缓冲，流控，并发，数据转换等核心技术

4.2.1.2 相关概念

DataX 完成单个数据同步的作业，称为 **Job**，DataX 接受到一个 Job 后，将启动一个 **进程** 完成整个作业同步过程。DataX Job 模块是单个作业的中枢管理节点，承担数据清理、子任务切分(将单一作业计算转化为多个子 Task)、TaskGroup 管理等功能

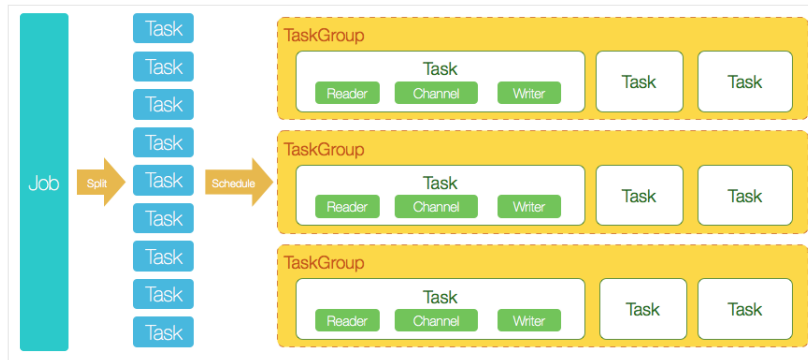
DataX Job 启动后，会根据不同的源端切分策略，将 Job 切分成多个小的 Task(子任务)，以便并发执行。**Task** 便是 DataX 作业的最小单元，每一个 Task 都会负责一部分数据的同步工作

切分多个 Task 之后，DataX Job 会调用 **Scheduler** 模块，根据配置的并发数据量，将拆分成的 Task 重新组合，组装成 **TaskGroup**(任务组)。每一个 TaskGroup 负责以一定的并发运行完毕分配好的所有 Task，默认单个任务组的并发数量为 5。

每一个 Task 都由 TaskGroup 负责启动，Task 启动后，会固定启动 Reader—>Channel—>Writer 的线程来完成任务同步工作

DataX 作业运行起来之后，Job 监控并等待多个 TaskGroup 模块任务完成，等待所有 TaskGroup 任务完成后 Job 成功退出。否则，异常退出，进程退出值非 0

涤生大数据



举例来说，用户提交了一个 DataX 作业，并且配置了 20 个并发，目的是将一个 100 张分表的 mysql 数据同步到 ods 里面。DataX 的调度决策思路是：

- 1) DataXJob 根据分库分表切分成了 100 个 Task。
- 2) 根据 20 个并发，DataX 计算共需要分配 4 个 TaskGroup。
- 3) 4 个 TaskGroup 平分切分好的 100 个 Task，每一个 TaskGroup 负责以 5 个并发共计运行 25 个 Task

4.2.1.3 安装

1) 下载 DataX 安装包并上传到 ds-bigdata-005 机器上的/opt/software，下载地址：
<http://datax-opensource.oss-cn-hangzhou.aliyuncs.com/datax.tar.gz>

2) 解压 datax.tar.gz 到当前的目录

3) 自检，执行如下命令

```
python datax.py ../job/job.json
```

出现如下内容，则表明安装成

```
[whujian888@ds-bigdata-005 bin]$ python datax.py ../job/job.json
DataX (DATA-OPEN-SOURCE-3.0), From Alibaba !
Copyright (C) 2010-2017, Alibaba Group. All Rights Reserved.

2023-11-19 11:30:47.771 [main] INFO MessageSource - JVM TimeZone: GMT+08:00, Locale: zh_CN
2023-11-19 11:30:47.773 [main] INFO MessageSource - use Locale: zh_CN timeZone: sun.util.calendar.ZoneInfo[id="C
lse,transitions=0,lastRule=null]
2023-11-19 11:30:47.782 [main] INFO VMInfo - VMInfo# operatingSystem class => sun.management.OperatingSystemImpl
2023-11-19 11:30:47.785 [main] INFO Engine - the machine info =>

osInfo: Oracle Corporation 1.8 25.181-b13
jvmInfo: Linux amd64 3.10.0-1160.62.1.el7.x86_64
cpu num: 16

totalPhysicalMemory: -0.00G
freePhysicalMemory: -0.00G
maxFileDescriptorCount: -1
currentOpenFileDescriptorCount: -1

GC Names [PS MarkSweep, PS Scavenge]

MEMORY_NAME | allocation_size | init_size
PS Eden Space | 256.00MB | 256.00MB
Code Cache | 240.00MB | 2.44MB
Compressed Class Space | 1,024.00MB | 0.00MB
PS Survivor Space | 42.50MB | 42.50MB
PS Old Gen | 683.00MB | 683.00MB
Metaspace | -0.00MB | 0.00MB
```

4.2.3 使用案例

4.2.3.1 使用前说明

任务提交命令：用户需要根据同步数据的数据源和目的地选择相应的 Reader 和 Writer，并将 Reader 和 Writer 的信息配置在一个 json 文件中，然后执行命令提交数据同步任务即可，即：**python datax.py youPath/job.json**

配置文件格式：查看 DataX 配置文件模板可以通过以下命令，如将 mysql 中的数据同步到 hdfs 中可以使用：**python datax.py -r mysqlreader -w hdfswriter**

```

{
  "job": {
    "content": [ 数据源和目的地相关配置
      {
        "reader": {  Reader相关配置
          "name": "mysqlreader",  Reader名称, 不可随意命名
          "parameter": { ... }  Reader配置参数
        },
        "writer": {  Writer相关配置
          "name": "hdfswriter",  Writer名称, 不可随意命名
          "parameter": { ... }  Writer配置参数
        }
      }
    ],
    "setting": {  Job配置参数, 包括限速配置等
      "speed": { ... }
    }
  }
}

```

json 最外层是一个 job，job 包含 setting 和 content 两部分，其中 setting 用于对整个 job 进行配置，content 用户配置数据源和目的地。Reader 和 Writer 的具体参数参考官方文档

<https://github.com/alibaba/DataX/blob/master/README.md>

<https://gitee.com/mirrors/DataX/blob/master/README.md>

类型	数据源	Reader(读)	Writer(写)	文档
RDBMS 关系型数据库	MySQL	√	√	读、写
	Oracle	√	√	读、写
	OceanBase	√	√	读、写
	SQLServer	√	√	读、写
	PostgreSQL	√	√	读、写
	DRDS	√	√	读、写
	通用RDBMS(支持所有关系型数据库)	√	√	读、写
阿里云数仓数据存储	ODPS	√	√	读、写
	ADS		√	写
	OSS	√	√	读、写
	OCS	√	√	读、写

4.2.3.1 读取 MySQL 中的数据存放到 HDFS

MySQLReader 具有两种模式，分别是 **TableMode** 和 **QuerySQLMode**，前者使用 table, column, where 等属性声明需要同步的数据；后者使用一条 SQL 查询语句声明需要同步的数据。

MySQLReader 之 TableMode

1) 编写配置文件

- (1) 创建配置文件：../job/stud_datax.json
- (2) 配置文件内容如 4E0B

```
{
  "job": {
    "content": [
      {
        "reader": {
          "name": "mysqlreader",
          "parameter": {
            "column": [
              "name",
              "age"
            ],
            "where": "name = 'Alice' ",
            "connection": [
              {
                "jdbcUrl": [
                  "jdbc:mysql://ds-bigdata-001:3306/ds_test"
                ],
                "table": [
                  "stud "
                ]
              }
            ],
            "password": "ds#readA0906",
            "splitPk": "",
            "username": "ds_read"
          }
        }
      ]
    }
  }
}
```

```

    }
  },
  "writer": {
    "name": "hdfswriter",
    "parameter": {
      "column": [
        {
          "name": "name",
          "type": "string"
        },
        {
          "name": "age",
          "type": "int"
        }
      ],
      "compress": "gzip",
      "defaultFS": "hdfs://ds-bigdata-001:8020",
      "fieldDelimiter": "\t",
      "fileName": "stud_datax",
      "fileType": "text",
      "path": "/test_demo/mysql_to_datax/",
      "writeMode": "append"
    }
  }
},
"setting": {
  "speed": {
    "channel": 1
  }
}
}
}

```

注意：HFDS Writer 并未提供 nullFormat 参数：也就是用户并不能自定义 null 值

写到 HDFS 文件中的存储格式。默认情况下，HDFS Writer 会将 null 值存储为空字符串（""），而 Hive 默认的 null 值存储格式为\N。所以后期将 DataX 同步的文件导入 Hive 表就会出现这个问题。

解决方案：

一是修改 DataX HDFS Writer 的源码，增加自定义 null 值存储格式的逻辑，参考 <https://blog.csdn.net/u010834071/article/details/105506580>

二是在 Hive 中建表时指定 null 值存储格式为空字符串（""），例如：

```
DROP TABLE IF EXISTS stud;
CREATE EXTERNAL TABLE stud
(
    `name`    STRING',
    `age`    INT
) COMMENT '学生表'
ROW FORMAT DELIMITED FIELDS TERMINATED BY '\t'
NULL DEFINED AS ''
LOCATION '/test_demo/stud';
```

2) 提交任务

(1) 在 HDFS 创建/test_demo/目录,使用 DataX 向 HDFS 同步数据时，需确保目标路径已存在

(2) 进入 DataX 的 bin 目录

(3) 执行命令 python datax.py ../job/stud_datax.json

4) 查看结果

(1) DataX 打印日志

```
2023-11-19 12:19:44.120 [job-0] INFO JobContainer -
[total cpu info] =>
averageCpu    | maxDeltaCpu    | minDeltaCpu
-1.00%        | -1.00%         | -1.00%

[total gc info] =>
NAME          | totalGCCount    | maxDeltaGCCount | minDeltaGCCount | totalGCtime    | maxDeltaGCtime  | minDeltaGCtime
PS MarkSweep  | 1               | 1               | 1               | 0.018s         | 0.018s          | 0.018s
PS Scavenge   | 1               | 1               | 1               | 0.009s         | 0.009s          | 0.009s

2023-11-19 12:19:44.120 [job-0] INFO JobContainer - PerfTrace not enable!
2023-11-19 12:19:44.120 [job-0] INFO StandAloneJobContainerCommunicator - Total 1 records, 7 bytes | Speed 0B/s, 0 records/s | Error 0 records, 0 bytes | All
aitWriterTime 0.000s | All Task WaitReaderTime 0.000s | Percentage 100.00%
2023-11-19 12:19:44.121 [job-0] INFO JobContainer -
任务启动时间    : 2023-11-19 12:19:32
任务结束时间    : 2023-11-19 12:19:44
任务总计耗时    : 11s
任务平均流量    : 0B/s
记录写入速度    : 0rec/s
读出记录总数    : 1
读写失败总数    : 0
```

(2) 查看 HDFS 文件

```
[whujian888@ds-bigdata-005 ~]$ hdfs dfs -ls /test_demo/mysql_to_datax/
Found 1 items
-rw-r--r-- 3 whujian888 supergroup 29 2023-11-19 12:19 /test_demo/mysql_to_datax/stud_datax_97ca3306_ee72_42ce_b5b3_562ca13d2ae1.gz
[whujian888@ds-bigdata-005 ~]$ hdfs dfs -cat /test_demo/mysql_to_datax/stud_datax_97ca3306_ee72_42ce_b5b3_562ca13d2ae1.gz | zcat
Alice 17
```

MySQLReader 之 QuerySQLMode

1) 编写配置文件

(1) 创建配置文件：../job/stud_datax.json

(2) 配置文件内容如下

```
{
  "job": {
    "content": [
      {
        "reader": {
          "name": "mysqlreader",
          "parameter": {
            "connection": [
              {
                "jdbcUrl": [
                  "jdbc:mysql://ds-bigdata-001:3306/ds_test"
                ],
                "querySql": [
                  "select name,age from stud where name =
'Alice' "
                ]
              }
            ],
            "password": "ds#readA0906",
            "username": "ds_read "
          }
        },
        "writer": {
          "name": "hdfswriter",
          "parameter": {
            "column": [
              {
                "name": "name",
                "type": "string"
              },
              {
                "name": "age",
                "type": "int"
              }
            ]
          }
        }
      }
    ]
  }
}
```

```

        },
    ],
    "compress": "gzip",
    "defaultFS": "hdfs://ds-bigdata-001:8020",
    "fieldDelimiter": "\t",
    "fileName": "stud_datax",
    "fileType": "text",
    "path": "/test_demo/mysql_to_datax/",
    "writeMode": "append"
    }
}
}
],
"setting": {
    "speed": {
        "channel": 1
    }
}
}
}

```

其他操作和 TableMode 中类似，这里就不在赘述。

4.2.3 DataX 的性能优化☆☆☆

4.2.3.1 速度控制

DataX3.0 提供了包括**通道(并发)**、**记录流**、**字节流**三种流控模式，可以随意控制作业速度，让作业在数据库可承受的范围内达到最佳的同步速度。

关键优化参数如下：

参数	说明
job.setting.speed.channel	并发数
job.setting.speed.record	总record限速 (tps每秒处理的条数)
job.setting.speed.byte	总byte限速 (bps每秒处理的字节数)
core.transport.channel.speed.record	单个channel的record限速，默认值为10000 (10000条/s)
core.transport.channel.speed.byte	单个channel的byte限速，默认值1024*1024 (1M/s)

注意事项:

- 1、若配置了总 record 限速，则必须配置单个 channel 的 record 限速
- 2、若配置了总 byte 限速，则必须配置单个 channel 的 byte 限速
- 3、若配置了总 record 限速和总 byte 限速，channel 并发数参数就会失效。因为配置了总 record 限速和总 byte 限速之后，实际 channel 并发数是通过计算得到的，计算公式为如下：

$\min(\text{总 byte 限速}/\text{单个 channel 的 byte 限速}, \text{总 record 限速}/\text{单个 channel 的 record 限速})$

配置示例：

```
{
  "core": {
    "transport": {
      "channel": {
        "speed": {
          "byte": 1048576 //单个 channel byte 限速 1M/s
        }
      }
    }
  },
  "job": {
    "setting": {
      "speed": {
        "byte": 5242880 //总 byte 限速 5M/s
      }
    }
  },
  ...
}
```

4.2.3.2 内存调整

当提升 DataX Job 内 Channel 并发数时，内存的占用会显著增加，因为 DataX 作为数据交换通道，在内存中会缓存较多的数据。例如 Channel 中会有一个 Buffer，作为临时的数据交换的缓冲区，而在部分 Reader 和 Writer 的中，也会存在一些

Buffer，为了防止 OOM 等错误，需调大 JVM 的堆内存。

建议将内存设置为 4G 或者 8G，这个也可以根据实际情况来调整。

调整 JVM xms xmx 参数的两种方式：一种是直接更改 datax.py 脚本；另一种是在启动的时候，加上对应的参数，如下：

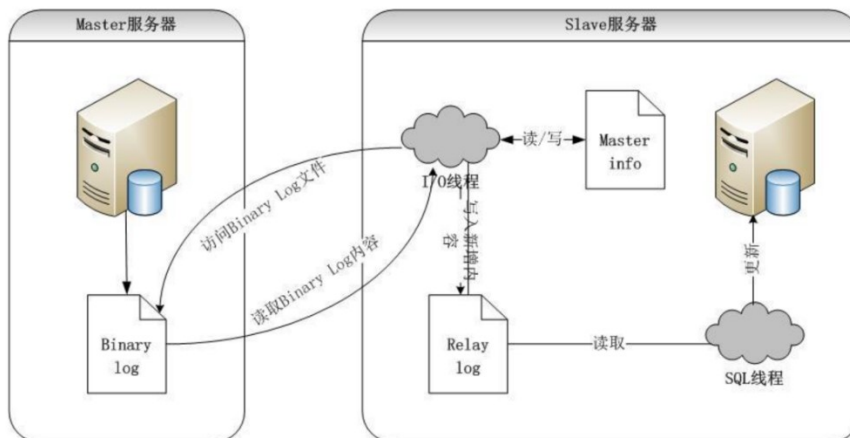
```
python datax.py --jvm="-Xms8G -Xmx8G" yourPath/job.json
```

4.3 实时-Maxwell☆☆

Maxwell 是由美国 Zendesk 开源，用 Java 编写的 **MySQL 实时抓取工具**。实时读取 MySQL 二进制日志 Binlog，并生成 JSON 格式的消息，作为生产者发送给 Kafka，Kinesis、RabbitMQ、Redis、Google Cloud Pub/Sub、文件或其它平台的应用程序。
官网地址：<http://maxwells-daemon.io/>

4.3.1 Maxwell 工作原理

Maxwell 的工作原理很简单，就是把自己伪装成 MySQL 的一个 slave，然后以 slave 的身份假装从 MySQL(master)复制数据。



MySQL 的二进制日志可以说 MySQL 最重要的日志了，它记录了所有的 DDL 和 DML(除了数据查询语句)语句，以事件形式记录，还包含语句所执行的消耗的时间，MySQL 的二进制日志是**事务安全型**的。

一般来说开启二进制日志大概会有 1% 的性能损耗。二进制有两个最重要的使用场景：

- 其一：MySQL Replication 在 Master 端开启 binlog，Master 把它的二进制日志传递给 slaves 来达到 master-slave 数据一致的目的。
- 其二：自然就是数据恢复了，通过使用 mysqlbinlog 工具来使恢复数据。

二进制日志包括两类文件：二进制日志索引文件（文件名后缀为 .index）用于记录所

有的二进制文件，二进制日志文件（文件名后缀为.000000*）记录数据库所有的 DDL 和 DML(除了数据查询语句)语句事件

4.3.2 Maxwell 与 Canal 的对比

Canal 是阿里巴巴旗下的一款开源项目，纯 Java 开发。基于数据库增量日志解析，提供增量数据订阅&消费，目前主要支持了 MySQL（也支持 mariaDB）。Canal 的原理和 Maxwell 的原理一模一样，都是通过伪装成 MySQL 的一个 slave，然后以 slave 的身份假装从 MySQL(master)复制数据。

对比	Canal	Maxwell
语言	java	java
数据格式	格式自由	json
采集数据模式	增量	全量/增量
数据落地	定制	支持 kafka 等多种平台
HA	支持	支持

4.3.3 Maxwell 使用

4.3.3.1 Maxwell 安装

Maxwell 官网地址: <http://maxwells-daemon.io/>

文档查看地址: <http://maxwells-daemon.io/quickstart/>

```
# 下载安装包
[whujian8888@ds-bigdata-005 software]$ wget
https://github.com/zendesk/maxwell/releases/download/v1.29.2/maxwell-
1.29.2.tar.gz
# 解压包
[whujian8888@ds-bigdata-005 software]$ tar -xf maxwell-1.40.5.tar.gz
# 目录结构
[whujian8888@ds-bigdata-005 maxwell]$ ls -al
Total 104
drwxr-xr-x 4 root root 4096 Sep 13 17:29 .
drwxr-xr-x 10 root root 4096 Sep 13 18:06 ..
drwxr-xr-x 2 root root 4096 Sep 13 17:29 bi
-rw-r--r-- 1 501 games 26503 Jun 11 01:23 config.md
```



```
-rw-r--r-- 1 501 games 12406 Nov 2 2022 config.properties.example
-rw-r--r-- 1 501 games 10259 Aug 27 12:08 kinesis-producer-
library.properties.example
drwxr-xr-x 3 501 games 20480 Sep 10 00:02 lib
-rw-r--r-- 1 501 games 548 Nov 30 2021 LICENSE
-rw-r--r-- 1 501 games 477 Nov 2 2022 log4j2.xml
-rw-r--r-- 1 501 games 3436 Sep 9 23:59 quickstart.md
-rw-r--r-- 1 501 games 1025 Sep 9 23:59 README.md
```

在 MySQL 中建立一个 maxwell 库用于存储 Maxwell 的元数据

```
CREATE DATABASE maxwell;
```

分配一个账号可以操作该数据库

```
GRANT ALL ON maxwell.* TO 'maxwell'@'%' IDENTIFIED BY '123456';
```

分配这个账号可以监控其他数据库的权限

```
GRANT SELECT ,REPLICATION SLAVE , REPLICATION CLIENT ON *.* TO ma
xwell@'%';
```

刷新 mysql 表权限

4.3.3.2 Maxwell 进程启动

Maxwell 进程启动方式有如下两种：

(1) 使用命令行参数启动 Maxwell 进程

```
./maxwell --user='root' --password='Admin_Root123' --host='ds-
bigdata-005' --producer=stdout
```

```
[whujian888@ds-bigdata-005 bin]$ ./maxwell --user='root' --password='Admin_Root123' --host='ds-bigdata-005' --producer=stdout
Using kafka version: 1.0.0
{"database":"wikidb","table":"scheduler_run_details","type":"insert","ts":1700392120,"xid":86788252,"commit":true,"data":{"id":10492542,"job_
_start_time":"2023-11-19 19:08:40.751000","duration":0,"outcome":"S","message":""}}
{"database":"openvpn_log","table":"access_type_info","type":"insert","ts":1700392122,"xid":86788256,"commit":true,"data":{"username":"yinmeihe
-11-19 19:08:42.000000","access_type":"login","trust_ip":"120.244.4.71","Remote_ip":"10.0.0.170","Rx":0,"Tx":0}}
{"database":"jumpserver","table":"terminal_command","type":"insert","ts":1700392124,"xid":86788290,"commit":true,"data":{"id":"3f0d4752800a422
":whujian888@whujian888"},"asset":"ds-bigdata-005","system_user":"ldap_login(whujian888)","input":"/maxwell --user='root'
3' --host= ds-bigdata-005 --producer=stdout","output":"/maxwell --user=root --password= Admin_Root123 --host= ds-bigdata-005 --producer=
rson: 1.0.0\n\n{"database":"wikidb","table":"scheduler_run_details","type":"insert","ts":1700392090,"xid":86787560,"commit":
0492525,"job_id":"SearchAuditListener"},"start_time":"2023-11-19 19:08:10.751000","duration":0,"outcome":"S","message":""}}
-445b-b1f-ed80f6fc232","timestamp":1700392087,"org_id":"00000000-0000-0000-0000-000000000002","risk_level":0}}
{"database":"wikidb","table":"scheduler_run_details","type":"insert","ts":1700392125,"xid":86788307,"commit":true,"data":{"id":10492543,"job_
or","start_time":"2023-11-19 19:08:45.000000","duration":1,"outcome":"S","message":""}}
{"database":"wikidb","table":"scheduler_run_details","type":"insert","ts":1700392125,"xid":86788337,"commit":true,"data":{"id":10492544,"job_
ue-broker-job-id","start_time":"2023-11-19 19:08:45.746000","duration":1,"outcome":"S","message":""}}
{"database":"wikidb","table":"scheduler_run_details","type":"insert","ts":1700392125,"xid":86788348,"commit":true,"data":{"id":10492545,"job_
_start_time":"2023-11-19 19:08:45.747000","duration":3,"outcome":"S","message":""}}
{"database":"wikidb","table":"scheduler_run_details","type":"insert","ts":1700392125,"xid":86788358,"commit":true,"data":{"id":10492546,"job_
_start_time":"2023-11-19 19:08:45.751000","duration":1,"outcome":"S","message":""}}
{"database":"wikidb","table":"scheduler_run_details","type":"insert","ts":1700392125,"xid":86788373,"commit":true,"data":{"id":10492547,"job_
kQueueFlusherSchedulerJob","start_time":"2023-11-19 19:08:45.767000","duration":1,"outcome":"S","message":""}}
{"database":"wikidb","table":"scheduler_run_details","type":"insert","ts":1700392125,"xid":86788403,"commit":true,"data":{"id":10492548,"job_
s.authentication.impl.basicAuth.job.UpdateBasicAuthConfigJob","start_time":"2023-11-19 19:08:45.775000","duration":1,"outcome":"S","messag
{"database":"wikidb","table":"scheduler_run_details","type":"insert","ts":1700392125,"xid":86788426,"commit":true,"data":{"id":10492549,"job_
_start_time":"2023-11-19 19:08:45.789000","duration":1,"outcome":"S","message":""}}
```

其中：--producer 生产者模式(stdout：控制台，kafka：kafka 集群)

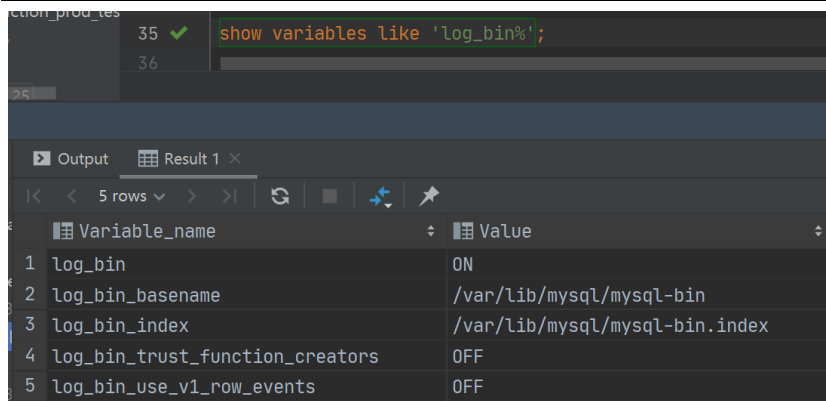
备注：在实操的时候发现以下错误，因为就是在 maxwell 1.30.0 开始，对 jdk 最低的要求已经是 11

```
[whujian888@ds-bigdata-005 bin]$ ./maxwell --user="ds_read" --password="ds_readA0906" --host="ds-bigdata-001" --producer=stdout
Error: A JNI error has occurred, please check your installation and try again
Exception in thread "main" java.lang.UnsupportedClassVersionError: com/zendesk/maxwell/Maxwell has been compiled by a more recent version of the Java Runtime (class file version 55.0); this version of the Java Runtime only recognizes class file versions up to 52.0
    at java.lang.ClassLoader.defineClass1(Native Method)
    at java.lang.ClassLoader.defineClass(ClassLoader.java:763)
    at java.security.SecureClassLoader.defineClass(SecureClassLoader.java:142)
    at java.net.URLClassLoader.defineClass(URLClassLoader.java:467)
    at java.net.URLClassLoader.access$000(URLClassLoader.java:72)
    at java.net.URLClassLoader$1.run(URLClassLoader.java:368)
    at java.net.URLClassLoader$1.run(URLClassLoader.java:362)
    at java.security.AccessController.doPrivileged(Native Method)
    at java.net.URLClassLoader.findClass(URLClassLoader.java:361)
    at java.lang.ClassLoader.loadClass(ClassLoader.java:424)
    at sun.misc.Launcher$MainClassLoader.loadClass(Launcher.java:349)
    at java.lang.ClassLoader.loadClass(ClassLoader.java:357)
    at sun.launcher.LauncherHelper.checkAndLoadMain(LauncherHelper.java:495)
[whujian888@ds-bigdata-005 bin]$ echo $JAVA_HOME
/usr/java/jdk1.8.0_181-cloudera
```

上述执行的前提是 mysql 已经开启 binlog，且相关的配置如下：

```
[client]
default_character_set=utf8

[mysqld]
server-id=1
collation_server=utf8_general_ci
character_set_server=utf8
log-bin=mysql-bin
binlog_format=row
expire_logs_days=30
```



The screenshot shows a MySQL command prompt with the command 'show variables like 'log_bin%';' entered. The output is displayed in a table with 5 rows and 2 columns: Variable_name and Value.

Variable_name	Value
log_bin	ON
log_bin_basename	/var/lib/mysql/mysql-bin
log_bin_index	/var/lib/mysql/mysql-bin.index
log_bin_trust_function_creators	OFF
log_bin_use_v1_row_events	OFF

(2) 修改配置文件，定制化启动 Maxwell 进 7A0B

```
[whujian888@ds-bigdata-005 bin]$ cp config.properties.example
config.properties
[whujian888@ds-bigdata-005 bin]$ vim config.properties
[whujian888@ds-bigdata-005 bin]$ bin/maxwell --
config ./config.properties
```

4.3.4 Maxwell 的缺点

1、对 jdk 的要求比较高，目前 maxwell 的版本是 1.40.6，但是从 1.30 开始对 jdk 的支持就已经上升到 11，而目前企业中用的最多的还是 jdk8，因此对企业的实际的生产环境有一定的要求

2、maxwell 对数据是无差别的监控，maxwell 的参数并没有提供类似于 database 和 table 这样的参数，意味着就监控这某个源的所有数据库和所有的表，因为不适用于大数据量的环境下，只适合使用数据量比较小的情况

3、只能读取 mysql 源，因此对于目前企业中多源异构的数据源的情况并不能做很好的支撑；并且对输出端的支持也不够多，如果要写入 hive 或者 hdfs 之中，需要借助 kafka 作为中间件进行转发

4.4 实时-Flink CDC☆☆

CDC 是 Change Data Capture (变更数据获取) 的简称。核心思想是，监测并捕获数据库的变动（包括数据或数据表的插入、更新以及删除等），将这些变更按发生的顺序完整记录下来（binlog），写入到消息中间件中以供其他服务进行订阅及消费。

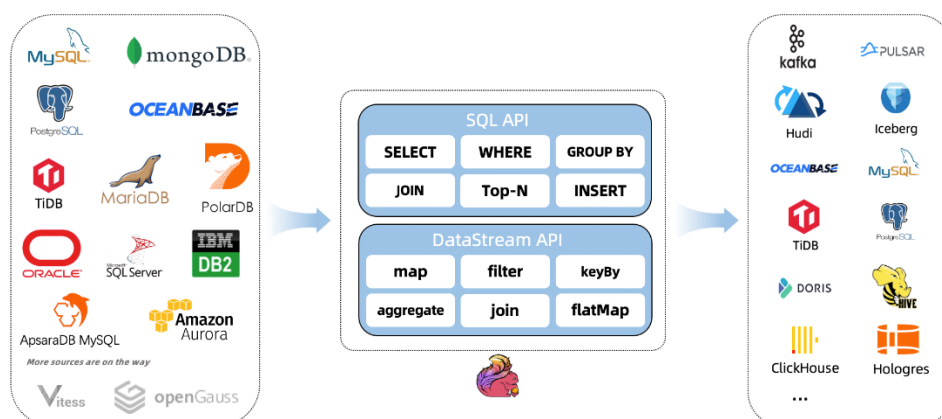
传统上，数据源的变化通常通过**周期性**地轮询整个数据集进行检查来实现。但是，这种轮询的方式效率低下且不能实时反应变化。而 CDC 技术则通过在数据源上设置一种机制，使得变化的数据可以被实时捕获并传递给下游处理系统，从而实现了实时的数据变动监控。。

CDC 主要分为基于查询和基于 Binlog 两种方式，先主要了解一下这两种之间的区别：

	基于查询的 CDC	基于 Binlog 的 CDC
开源产品	Sqoop、Kafka JDBC Source	Canal、Maxwell、Debezium
执行模式	Batch	Streaming
是否可以捕获所有数据变化	否	是
延迟性	高延迟	低延迟
是否增加数据库压力	是	否

Flink 社区开发了 flink-cdc-connectors 组件，这是一个可以直接从 MySQL、PostgreSQL 等数据库直接**读取全量数据和增量**变更数据的 source 组件。目前也已开源，开源地址：<https://github.com/ververica/flink-cdc-connectors>。

使用 Flink CDC，我们可以轻松地构建实时数据管道，对数据变动进行实时响应和处理，为实时分析、实时报表和实时决策等场景提供强大的支持 3002



4.4.1 flink 与 flink cdc 的版本对应

Flink CDC版本	Flink 版本_
1.0.0	1.11.*
1.1.0	1.11.*
1.2.0	1.12.*
1.3.0	1.12.*
1.4.0	1.13.*
2.0.*	1.13.*
2.1.*	1.13.*
2.2.*	1.13.*, 1.14.*
2.3.*	1.13.*, 1.14.*, 1.15.*, 1.16.0
2.4.*	1.13.*, 1.14.*, 1.15.*, 1.16.*, 1.17.0

4.4.2 CDC 实现的原理

通常来讲，CDC 分为**主动查询**和**事件接收**两种技术实现模式。

对于主动查询而言，用户通常会在数据源表的**某个字段中，保存上次更新的时间戳或版本号等信息**，然后下游通过不断的查询和与上次的记录做对比，来确定数据是否有变动，是否需要同步。

优点：不涉及数据库底层特性，实现比较通用；

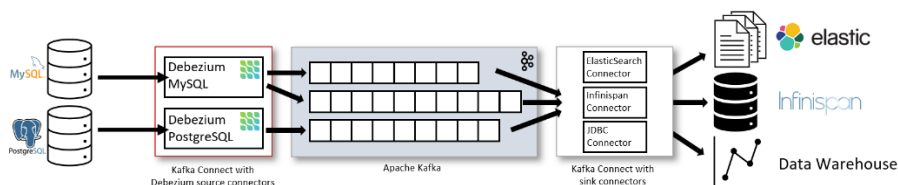
缺点：要对业务表做改造，且实时性不高，不能确保跟踪到所有的变更记录，且持续的频繁查询对数据库的压力较大。

事件接收模式可以通过触发器（Trigger）或者日志（例如 Transaction log、Binary log、Write-ahead log 等）来实现。当数据源表发生变动时，会通过附加在表上的触发器或者 binlog 等途径，将操作记录下来。下游可以通过**数据库底层的协议**，订阅并消费这些事件，然后对数据库变动记录做**重放**，从而实现同步。

优点：实时性高，可以精确捕捉上游的各种变动；

缺点：部署数据库的事件接收和**解析器（例如 Debezium、Canal、Maxwell 等）**，有一定的学习和运维成本，对一些冷门的数据库支持不够。

综合来看，事件接收模式整体在实时性、吞吐量方面占优，如果数据源是 MySQL、PostgreSQL、MongoDB 等常见的数据库实现，建议使用 **Debezium** 来实现变更数据的捕获（下图来自 Debezium 官方文档如果使用的只有 MySQL，则还可以用 Canal。



4.4.3 为什么选择 Flink

从上图可以看到，Debezium 官方架构图中，是通过 Kafka Streams 直接实现的 CDC 功能。而我们这里更建议使用 Flink CDC 模块，因为 Flink 相对 Kafka Streams 而言，有如下优势 FF1A

强大的流处理引擎：Flink 是一个强大的流处理引擎，具备**高吞吐量、低延迟、Exactly-Once** 语义等特性。它通过基于事件时间的处理模型，支持准确和有序的数据处理，适用于实时数据处理和分析场景。这使得 Flink 成为实现 CDC 的理想选择。

内置的 CDC 功能：Flink 提供了内置的 CDC 功能，可以直接连接到各种数据源，捕获数据变化，并将其作为数据流进行处理。这消除了我们自行开发或集成 CDC 解决方案的需要，使得实现 CDC 变得更加简单和高效，开箱即用。

多种数据源的支持：Flink CDC 支持与各种数据源进行集成，如关系型数据库（如 MySQL、PostgreSQL）、消息队列（如 Kafka、RabbitMQ）、文件系统等。这意味着无论你的数据存储在哪里，Flink 都能够轻松地捕获其中的数据变化，并进行进一步的实时处理和分析。

灵活的数据处理能力：Flink 提供了灵活且强大的数据处理能力，可以通过编写自定义的转换函数、处理函数等来对 CDC 数据进行各种实时计算和分析。同时，Flink 还集成了 SQL 和 Table API，为用户提供了使用 SQL 查询语句或 Table API 进行简单查询和分析的方式

完善的生态系统：Flink 拥有活跃的社区和庞大的生态系统，这意味着你可以轻松地获取到丰富的文档、教程、示例代码和解决方案。此外，Flink 还与其他流行的开源项目（如 Apache Kafka、Elasticsearch）深度集成，提供了更多的功能和灵活性。

4.4.4 Flink CDC 特性

1. 支持读取数据库快照，即使出现故障也能继续读取 binlog，并进行 **Exactly-once** 处理
2. DataStream API 的 CDC 连接器，用户可以在单个作业中使用多个数据库和表的更改，而无需部署 Debezium 和 Kafka
3. Table/SQL API 的 CDC 连接器，用户可以使用 SQL DDL 创建 CDC 源来监视单个表上的更改。

下表显示了连接器的当前特性 FF1A

涤生大数据

连接器	无锁读	并行读	一次性语义读	增量快照读
MongoDB-CDC	✓	✓	✓	✓
mysql-cdc	✓	✓	✓	✓
Oracle-CDC	✓	✓	✓	✓
Postgres-CDC	✓	✓	✓	✓
sqlserver-cdc	✓	✓	✓	✓
Oceanbase-CDC	✗	✗	✗	✗
TiDB-CDC	✓	✗	✓	✗
db2-cdc	✗	✗	✓	✗
vitess-cdc	✓	✗	✓	✗

4.4.5 用法实例

在使用 flink cdc 开发数据同步时，需要一定的编程基础，相关的依赖如下：

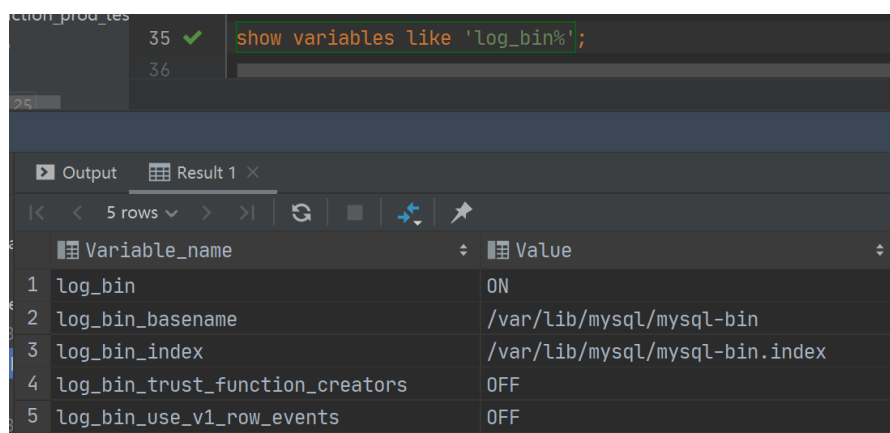
```
<properties>
  <flink.version>1.13.0</flink.version>
  <maven.compiler.source>1.8</maven.compiler.source>
  <maven.compiler.target>1.8</maven.compiler.target>
</properties>
<dependency>
  <groupId>com.ververica</groupId>
  <artifactId>flink-connector-mysql-cdc</artifactId>
  <version>2.2.0</version>
</dependency>
<dependency>
  <groupId>org.apache.flink</groupId>
  <artifactId>flink-connector-jdbc_2.12</artifactId>
  <version>${flink.version}</version>
</dependency>
<!-- flink 核心 API -->
<dependency>
  <groupId>org.apache.flink</groupId>
  <artifactId>flink-clients_2.12</artifactId>
  <version>${flink.version}</version>
</dependency>
<dependency>
```

```
<groupId>org.apache.flink</groupId>
<artifactId>flink-java</artifactId>
<version>${flink.version}</version>
</dependency>
<dependency>
  <groupId>org.apache.flink</groupId>
  <artifactId>flink-scala_2.12</artifactId>
  <version>${flink.version}</version>
</dependency>
<dependency>
  <groupId>org.apache.flink</groupId>
  <artifactId>flink-streaming-java_2.12</artifactId>
  <version>${flink.version}</version>
</dependency>
<dependency>
  <groupId>org.apache.flink</groupId>
  <artifactId>flink-streaming-scala_2.12</artifactId>
  <version>${flink.version}</version>
</dependency>
<dependency>
  <groupId>org.apache.flink</groupId>
  <artifactId>flink-table-common</artifactId>
  <version>${flink.version}</version>
</dependency>
<dependency>
  <groupId>org.apache.flink</groupId>
  <artifactId>flink-table-planner-blink_2.12</artifactId>
  <version>${flink.version}</version>
</dependency>
<dependency>
  <groupId>org.apache.flink</groupId>
  <artifactId>flink-table-api-java-bridge_2.12</artifactId>
  <version>${flink.version}</version>
</dependency>
```

并在开发前，mysql 已经开启 binlog，且相关的配置如下：

```
[client]
default_character_set=utf8

[mysqld]
server-id=1
collation_server=utf8_general_ci
character_set_server=utf8
log-bin=mysql-bin
binlog_format=row
expire_logs_days=30
```



The screenshot shows a MySQL command prompt with the command `show variables like 'log_bin%';` entered. Below the command, the output is displayed as a table with 5 rows and 2 columns: Variable_name and Value.

	Variable_name	Value
1	log_bin	ON
2	log_bin_basename	/var/lib/mysql/mysql-bin
3	log_bin_index	/var/lib/mysql/mysql-bin.index
4	log_bin_trust_function_creators	OFF
5	log_bin_use_v1_row_events	OFF

4.4.5.1 DataStream API 的用法

```
public class CDCDemo {
    public static void main(String[] args) throws Exception {

        StreamExecutionEnvironment env =
            StreamExecutionEnvironment.getExecutionEnvironment();

        env.setParallelism(1);
        env.enableCheckpointing(3000);

        DebeziumSourceFunction<String> sourceFunction =
            MySqlSource.<String> builder()
                .hostname("ds-bigdata-005")
                .port(3306)
```



```

        .username("root")
        .password("Admin_Root123")
        .databaseList("lili")
        // 这里一定要是 db.table 的形式
        .tableList("lili.test_cdc")
        .deserializer(new StringDebeziumDeserializationSchema())
        .startupOptions(StartupOptions.initial())
        .build();

    DataStreamSource<String> dataStreamSource =
env.addSource(sourceFunction);
    dataStreamSource.print();
    env.execute("CDCDemo");
}
}

```

4.4.5.2 Flink SQL API 的用法

```

public class FlinkSQLCDC {
    public static void main(String[] args) throws Exception {

        StreamExecutionEnvironment env =
StreamExecutionEnvironment.getExecutionEnvironment();
        env.setParallelism(1);
        env.enableCheckpointing(3000);

        StreamTableEnvironment tableEnv =
StreamTableEnvironment.create(env);
        tableEnv.executeSql("CREATE TABLE test_cdc (" +
            " id int primary key," +
            " name STRING," +
            " age int" +
            ") WITH (" +

```

```

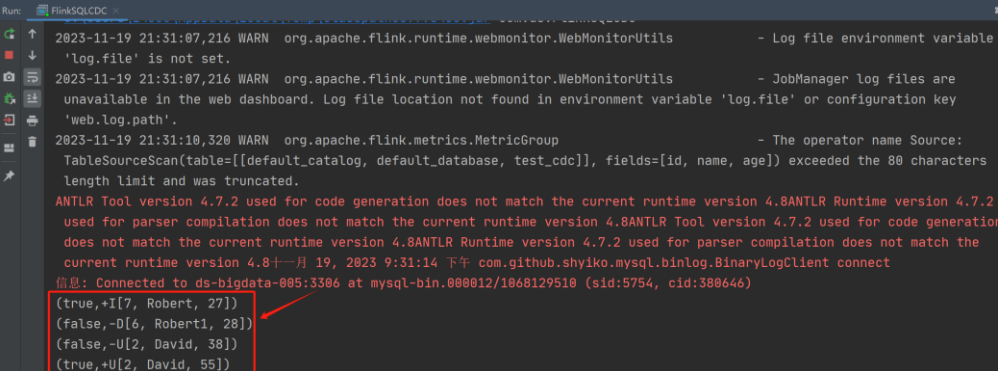
    " 'connector' = 'mysql-cdc'," +
    " 'scan.startup.mode' = 'latest-offset'," +
    " 'hostname' = 'ds-bigdata-005'," +
    " 'port' = '3306'," +
    " 'username' = 'root'," +
    " 'password' = 'Admin_Root123'," +
    " 'database-name' = 'lili'," +
    " 'table-name' = 'test_cdc'" +
    ");";

Table table = tableEnv.sqlQuery("select * from test_cdc");
DataStream<Tuple2<Boolean, Row>> dataStreamSource =
tableEnv.toRetractStream(table, Row.class);

dataStreamSource.print();

env.execute("FlinkSQLCDC");
}
}

```



```

Run: FlinkSQLCDC
2023-11-19 21:31:07,216 WARN org.apache.flink.runtime.webmonitor.WebMonitorUtils - Log file environment variable
'log.file' is not set.
2023-11-19 21:31:07,216 WARN org.apache.flink.runtime.webmonitor.WebMonitorUtils - JobManager log files are
unavailable in the web dashboard. Log file location not found in environment variable 'log.file' or configuration key
'web.log.path'.
2023-11-19 21:31:10,320 WARN org.apache.flink.metrics.MetricGroup - The operator name Source:
TableSourceScan(table=[[default_catalog, default_database, test_cdc]], fields=[id, name, age]) exceeded the 80 characters
length limit and was truncated.
ANTLR Tool version 4.7.2 used for code generation does not match the current runtime version 4.8ANTLR Runtime version 4.7.2
used for parser compilation does not match the current runtime version 4.8ANTLR Tool version 4.7.2 used for code generation
does not match the current runtime version 4.8ANTLR Runtime version 4.7.2 used for parser compilation does not match the
current runtime version 4.8十一月 19, 2023 9:31:14 下午 com.github.shyiko.mysql.binlog.BinaryLogClient connect
信息: Connected to ds-bigdata-005:3306 at mysql-bin.000012/1060129510 (sid:5754, cid:380646)
(true,+I[7, Robert, 27])
(false,-D[6, Robert1, 28])
(false,-U[2, David, 38])
(true,+U[2, David, 55])

```

4.4.5.3 生产案例：MySQLToMySQL

```

public class MySQLToMySQL {
    public static void main(String[] args) throws Exception {

        StreamExecutionEnvironment env =
StreamExecutionEnvironment.getExecutionEnvironment();

        env.setParallelism(1);

        env.enableCheckpointing(3000);
    }
}

```

```

StreamTableEnvironment tableEnv =
StreamTableEnvironment.create(env);
// 如果原表有主键，则需要声明主键
tableEnv.executeSql("CREATE TABLE my_source (\n" +
    "    id INT primary key,\n" +
    "    name STRING,\n" +
    "    age INT\n" +
    ") WITH (\n" +
    "    'connector' = 'mysql-cdc',\n" +
    "    'hostname' = 'ds-bigdata-005',\n" +
    "    'port' = '3306',\n" +
    "    'username' = 'root',\n" +
    "    'password' = 'Admin_Root123',\n" +
    "    'scan.startup.mode' = 'initial'," +
    "    'database-name' = 'lili',\n" +
    "    'table-name' = 'test_cdc',\n" +
    "    'scan.incremental.snapshot.enabled' = 'false'\n" +
    ");");

// 表必须要提前创建号
tableEnv.executeSql("CREATE TABLE my_sink (\n" +
    "    id INT primary key,\n" +
    "    name STRING,\n" +
    "    age INT\n" +
    ") WITH (\n" +
    "    'connector' = 'jdbc',\n" +
    "    'url' = 'jdbc:mysql://ds-bigdata-005:3306/lili?serverTimezone=UTC&useUnicode=true&characterEncoding=utf-8&useSSL=false',\n" +
    "    'username' = 'root',\n" +
    "    'password' = 'Admin_Root123',\n" +
    "    'driver' = 'com.mysql.cj.jdbc.Driver',\n" +
    "    'table-name' = 'test_cdc1'\n" +

```

```

        "    ");

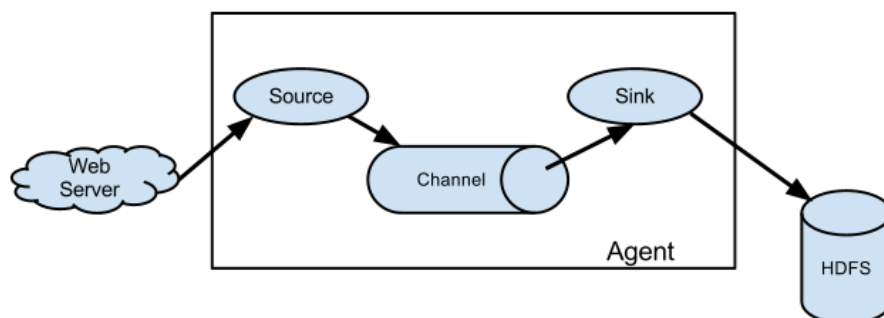
tableEnv.executeSql("INSERT INTO my_sink\n" +
    "    SELECT id, name, age\n" +
    "    FROM my_source");
    }
}

```

4.5 文件同步-Flume☆☆

4.5.1 相关概念

Flume 是 Cloudera 提供的一个高可用的，高可靠的，分布式的海量日志采集、聚合和传输的系统。Flume 基于流式架构，灵活简单。Flume 最主要的作用就是，实时读取服务器本地磁盘的数据，将数据写入到 HDFS 等 3002



Agent 是一个 JVM 进程，它以事件的形式将数据从源头送至目的。Agent 主要有 3 个部分组成，Source、Channel、Sink。

Source 是负责接收数据到 Flume Agent 的组件。Source 组件可以处理各种类型、各种格式的日志数据，包括 avro、thrift、exec、jms、spooling directory、netcat、taildir、sequence generator、syslog、http、legacy。**在企业中使用最广泛的就是日志文件，企业中重点使用两个 Source：SpoolingDirSource 和 TailDirSource**

SpoolingDirSource：监控一个目录，读取目录中新增的文件，将文件的内容封装为 event。

TailDirSource：监控一个或者多个文件，读取多个文件**最新追加**写入的内容。不会丢数据（可靠的）即使 flume 出现了故障或挂掉。Taildir Source 在工作时，会将读取文件的最后的位置记录在一个 json 文件中，一旦 agent 重启，会从之前已经记录的位置，继续执行 tail 操作。Json 文件中位置是可以修改，修改后 Taildir Source 会从修改的位置

进行 tail 操作。如果 JSON 文件丢失了，此时会重新从每个文件的第一行重新读取，这会造成数据的重复。TailDirSource 采集的文件，不能随意重命名。如果日志正在写入时，名称为 xxxx.tmp，写入完成后改名为 xxx.log，此时一旦匹配规则可以匹配上述名称，就会发生数据的重复采集。

Channel 是位于 Source 和 Sink 之间的缓冲区。因此，Channel 允许 Source 和 Sink 运作在不同的速率上。Channel 是线程安全的，可以同时处理几个 Source 的写入操作和几个 Sink 的读取操作。

Flume 自带三种 Channel：Memory Channel、File Channel 及 Kafka Channel。Memory Channel 是内存中的队列。**Memory Channel 在不需要关心数据丢失的情景下适用。如果需要关心数据丢失，那么 Memory Channel 就不应该使用，因为程序死亡、机器宕机或者重启都会导致数据丢失。**File Channel 将所有事件写到磁盘。因此在程序关闭或机器宕机的情况下不会丢失数据。

Flume+Kafka 联合优势：

可靠性：Flume 可以将数据从不同 source 采集，并通过 Kafka Channel 传输到 Kafka，确保数据不丢失。Kafka 本身也提供了副本机制来保证数据的可靠性。

弹性伸缩：由于 Flume 和 Kafka 都是分布式系统，它们可以根据需求进行横向扩展，以适应大规模数据传输和处理的需求。

高吞吐量：Kafka 的高吞吐量特性可以满足大量数据的传输需求，而 Flume 提供了丰富的 sources 和 sinks，使得数据流转更加灵活和高效。

Sink 不断地轮询 Channel 中的事件（Event，每一条记录都叫做一个事件）且批量地移除它们，并将这些事件批量写入到存储或索引系统、或者被发送到另一个 Flume Agent。Sink 组件目的地包括 hdfs、logger、avro、thrift、ipc、file、HBase、solr、自定义。

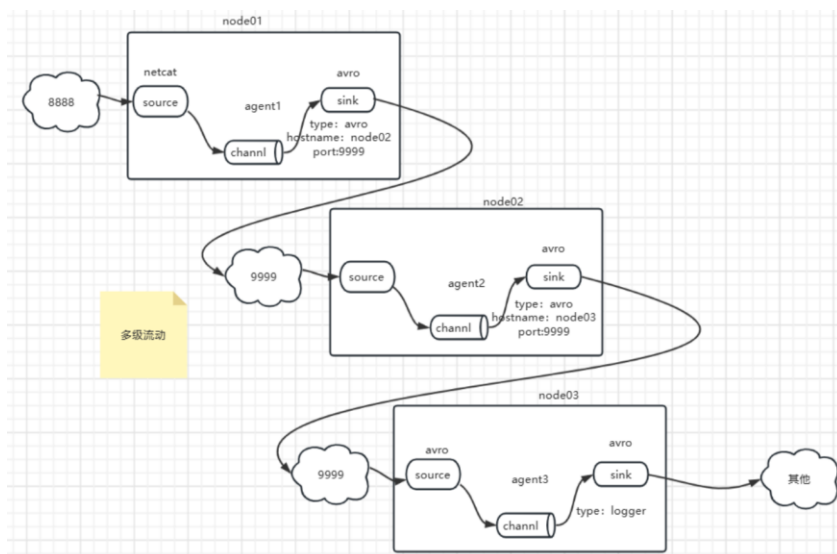
备注：Flume0.9X：又称 Flume-og，老版本的 flume，需要引入 zookeeper 集群管理，性能也较低（单线程工作）】

Flume1.X：又称 Flume-ng。新版本需要引入 zookeeper，和 flume-og 不兼容
除了上面说的 source->channel->sink 的流程，其实在 flume 中支持多种组合的流程，如下

多级联动：

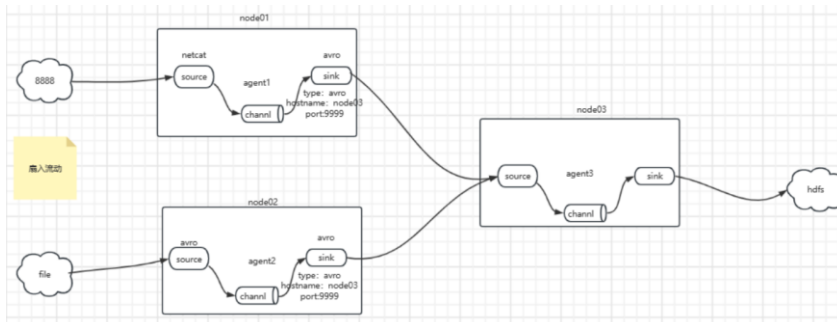
多级流动是指数据通过多个 Flume 代理（Agent）传输的模式。每个 Agent 在数据传输过程中既可以作为上游代理（**Upstream Agent**），又可以作为下游代理（**Downstream Agent**）。上游 agent 将数据发送到下游 agent 的 Source，并通过 Channel 进行缓冲和存储，然后由下游 agent 的 Sink 将数据传输到目标系统。这种模式可以用于构建复杂的数据流水线，使数据在多个 agent 之间进行

分发和处理。



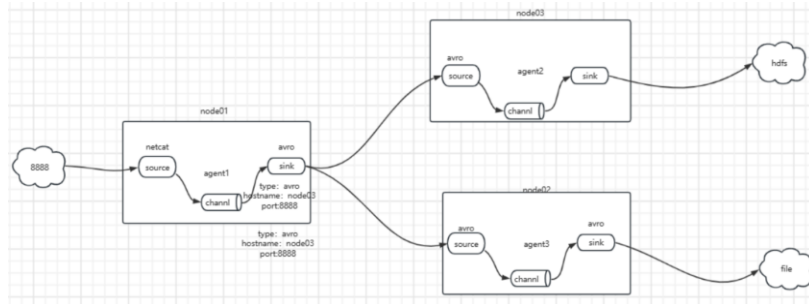
扇入流动:

扇入流动是指多个上游数据源将数据发送到同一个下游 agent 的模式。不同的数据源可以是不同的 Flume agent，也可以是同一个 agent 的不同 Source。下游 agent 的 Source 负责接收和收集来自多个数据源的数据，通过 Channel 进行缓冲和存储，然后将数据传输到目标系统的 Sink。扇入流动可以用于**聚合**来自不同数据源的数据，将其合并成一个数据流，然后进行进一步的处理或存储。



扇出流动:

扇出流动是指一个上游数据源将数据发送到多个下游 agent 的模式。上游数据源可以是一个 Flume agent 的 Source，而下游 agent 可以是不同的 Flume agent 或同一个 agent 的不同 Sink。上游数据源通过 Channel 将数据发送到多个下游代理的 Sink，从而实现数据的**复制或分发**。扇出流动可以用于将同一份数据发送到不同的目标系统，以实现数据备份、冗余或并行处理



4.5.2 企业典型案例

4.5.2.1 模拟端口发送数据

1、编写 agent 文件 (netcat-logger.conf)

```
# 定义这个 agent 中各组件的名字，前面的 a1 是 agent 的名字，该名字可以随便命名
a1.sources = r1
a1.sinks = k1
a1.channels = c1

# 描述和配置 source 组件: r1
a1.sources.r1.type = netcat
a1.sources.r1.bind = 0.0.0.0
a1.sources.r1.port = 9999

# 描述和配置 sink 组件: k1
a1.sinks.k1.type = logger

# 描述和配置 channel 组件，此处使用是内存缓存的方式
a1.channels.c1.type = memory
a1.channels.c1.capacity = 1000
a1.channels.c1.transactionCapacity = 100

# 组装
# source 和 channel 关联
a1.sources.r1.channels = c1
```

```
# sink 也要关联 channel
```

```
a1.sinks.k1.channel = c1
```

2、启动 flume-ng 服务

```
flume-ng agent -n a1 -c conf -f netcat-logger.conf -
```

```
Dflume.root.logger=INFO,console
```

3、使用 telnet 工具模拟客户端发送数据，然后就可以在界面看到发送过来的数据

```
acme.flume.Channel.MemoryChannel{name: c1} }
23/11/20 00:19:08 INFO node.Application: Starting Channel c1
23/11/20 00:19:08 INFO instrumentation.MonitoredCounterGroup: Monitored counter group for type: CHANNEL, name: c1: Successful
23/11/20 00:19:08 INFO instrumentation.MonitoredCounterGroup: Component type: CHANNEL, name: c1 started
23/11/20 00:19:08 INFO node.Application: Starting Sink k1
23/11/20 00:19:08 INFO node.Application: Starting Source r1
23/11/20 00:19:08 INFO source.NetcatSource: Source starting
23/11/20 00:19:08 INFO source.NetcatSource: Created serverSocket:sun.nio.ch.ServerSocketChannelImpl[/0.0.0.0:9875]
23/11/20 00:20:04 INFO sink.LoggerSink: Event: { headers:{} body: 6E 69 68 61 6F 0D nihao. }
23/11/20 00:20:06 INFO sink.LoggerSink: Event: { headers:{} body: 68 61 68 0D hah. }
23/11/20 00:20:21 INFO sink.LoggerSink: Event: { headers:{} body: 64 08 0D d.. }
```

4、可以添加参数在页面上看到 flume 相关的 metrics。

```
flume-ng agent -n a1 \
```

```
-c conf \
```

```
-f netcat-logger.conf \
```

```
-Dflume.monitoring.type=http \
```

```
-Dflume.monitoring.port=34345 \
```

```
-Dflume.root.logger=INFO,console
```

4.5.2.2 监控文件的变化写到 HDFS 中

1、编写 agent 文件 (tail-to-hdfs.conf)

```
# Name the components on this agent
```

```
taildir-hdfs-agent.sources = taildir-source
```

```
taildir-hdfs-agent.sinks = hdfs-sink
```

```
taildir-hdfs-agent.channels = memory-channel
```

```
# Describe/configure the source
```

```
taildir-hdfs-agent.sources.taildir-source.type = TAILDIR
```

```
taildir-hdfs-agent.sources.taildir-source.filegroups = f1
```

```
taildir-hdfs-agent.sources.taildir-source.filegroups.f1 =
```

```
/opt/software/flume-data/.*
```

```
taildir-hdfs-agent.sources.taildir-source.positionFile =
```

```
/opt/software/taildir/taildir_position.json
```



```
# Describe the sink
taildir-hdfs-agent.sinks.hdfs-sink.type = hdfs
taildir-hdfs-agent.sinks.hdfs-sink.hdfs.path = hdfs://ds-bigdata-001:8020/tmp/taildir/%Y%m%d%H%M
taildir-hdfs-agent.sinks.hdfs-sink.hdfs.useLocalTimeStamp = true
taildir-hdfs-agent.sinks.hdfs-sink.hdfs.fileType = CompressedStream
taildir-hdfs-agent.sinks.hdfs-sink.hdfs.writeFormat = Text
taildir-hdfs-agent.sinks.hdfs-sink.hdfs.codec = gzip
taildir-hdfs-agent.sinks.hdfs-sink.hdfs.filePrefix = wsk
taildir-hdfs-agent.sinks.hdfs-sink.hdfs.rollInterval = 30
taildir-hdfs-agent.sinks.hdfs-sink.hdfs.rollSize = 1024
taildir-hdfs-agent.sinks.hdfs-sink.hdfs.rollCount = 0

# Use a channel which buffers events in memory
taildir-hdfs-agent.channels.memory-channel.type = memory
taildir-hdfs-agent.channels.memory-channel.capacity = 1000
taildir-hdfs-agent.channels.memory-channel.transactionCapacity = 100

# Bind the source and sink to the channel
taildir-hdfs-agent.sources.taildir-source.channels = memory-channel
taildir-hdfs-agent.sinks.hdfs-sink.channel = memory-channel
```

2、启动 flume-ng 服务

```
flume-ng agent --conf conf --conf-file taildir-to-hdfs.conf --name taildir-hdfs-agent -Dflume.root.logger=INFO,console
```

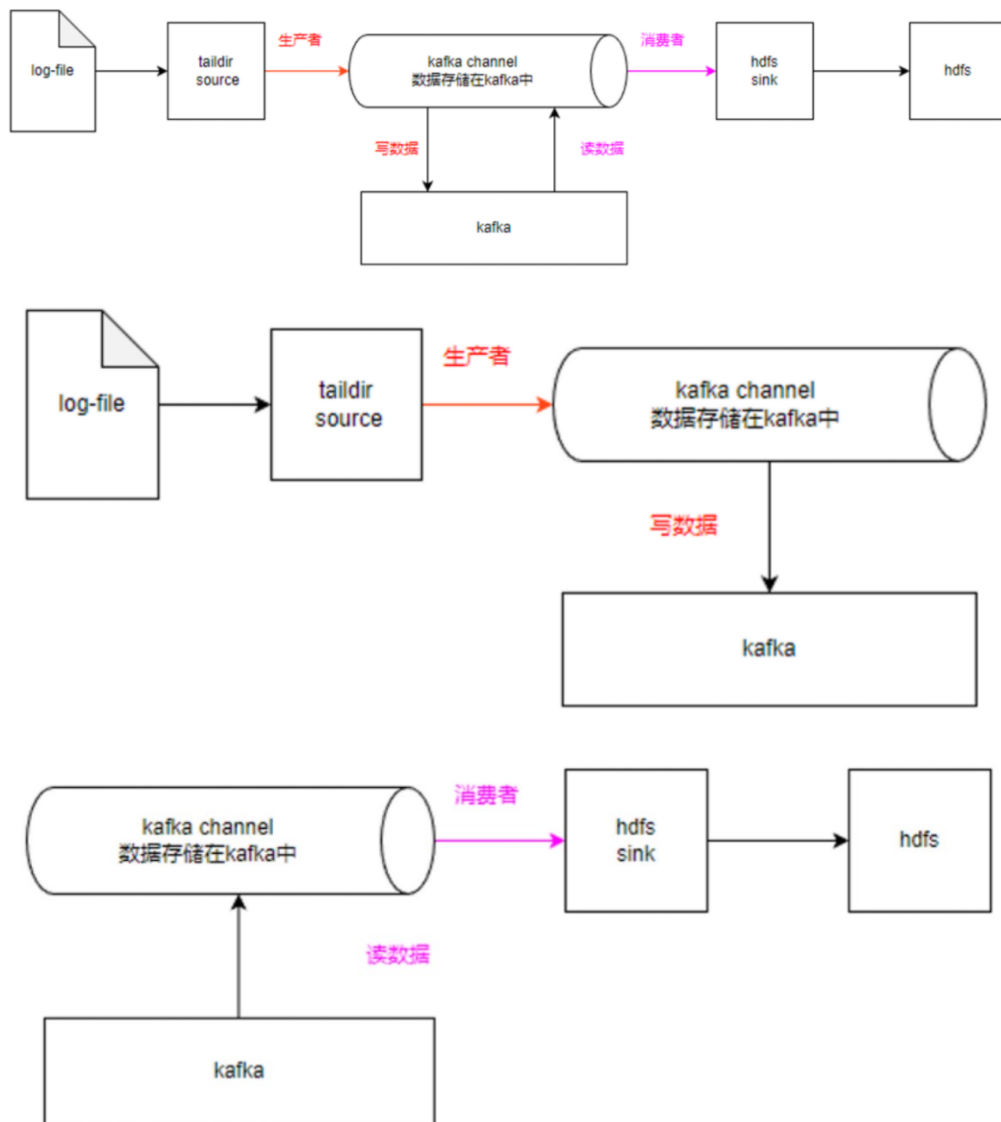
3、观察现象

```
23/11/20 00:41:10 INFO hdfs.HDFSEventSink: Writer callback called.
23/11/20 00:41:10 INFO hdfs.BucketWriter: Closing hdfs://ds-bigdata-001:8020/tmp/taildir/202311200040/wsk.1700412040225.gz.tmp
23/11/20 00:41:10 INFO hdfs.BucketWriter: Renaming hdfs://ds-bigdata-001:8020/tmp/taildir/202311200040/wsk.1700412040225.gz.tmp to hdfs://ds-bigdata-001:8020/tmp/taildir/202311200040/wsk.1700412040225.gz
23/11/20 00:42:21 INFO taildir.ReliableTaildirEventReader: Opening file: /opt/software/flume-data/tair-to-hdfs.conf, inode: 23330822, pos: 0
23/11/20 00:42:25 INFO hdfs.HDFSCompressedDataStream: Serializer = TEXT, UseLocalFileSystem = false
23/11/20 00:42:25 INFO hdfs.BucketWriter: Creating hdfs://ds-bigdata-001:8020/tmp/taildir/202311200042/wsk.1700412145918.gz.tmp
23/11/20 00:42:25 INFO hdfs.BucketWriter: Closing hdfs://ds-bigdata-001:8020/tmp/taildir/202311200042/wsk.1700412145918.gz.tmp
23/11/20 00:42:25 INFO hdfs.BucketWriter: Renaming hdfs://ds-bigdata-001:8020/tmp/taildir/202311200042/wsk.1700412145918.gz.tmp to hdfs://ds-bigdata-001:8020/tmp/taildir/202311200042/wsk.1700412145918.gz
23/11/20 00:42:25 INFO hdfs.BucketWriter: Creating hdfs://ds-bigdata-001:8020/tmp/taildir/202311200042/wsk.1700412145919.gz.tmp

[root@ds-bigdata-005 software]# hdfs dfs -ls /tmp/taildir
Found 2 items
drwxrwxrwx - root supergroup 0 2023-11-20 00:41 /tmp/taildir/202311200040
drwxrwxrwx - root supergroup 0 2023-11-20 00:42 /tmp/taildir/202311200042
[root@ds-bigdata-005 software]#
[root@ds-bigdata-005 taildir]# cat taildir.position.json
[{"inode":23330821,"pos":550,"file":"/opt/software/flume-data/netcat-logger.conf"}, {"inode":23330822,"pos":1540,"file":"/opt/software/flume-data/tair-to-hdfs.conf"}]
```

4.5.2.3 关于 KafkaChannel

KafkaChannel 可以看作是 MemeoryChannel 和 FileChannel 的折中版本，既兼顾了 FileChannel 的持久化数据，又兼顾了 Memeory 的处理性能，但是又摒弃了二者的缺点，可以说 KafkaChannel 是目前企业中用的最多的 Channel。常见的拓扑结构有以下三种：



案例：监控一个文件的变化，然后通过 kafka 输入

```
#定义 agent 名， source、channel 的名称
a0.sources = r1
a0.channels = c1

#具体定义 source
a0.sources.r1.type = exec
```

```
a0.sources.r1.command = tail -F /opt/software/logs.txt

a0.channels.c1.type = org.apache.flume.channel.kafka.KafkaChannel
a0.channels.c1.brokerList = ds-bigdata-002:9092,ds-bigdata-003:9092,ds-
bigdata-004:9092
a0.channels.c1.zookeeperConnect=ds-bigdata-001:2181/kafka
a0.channels.c1.topic = test-wang
#false 表示是以纯文本的形式写进入的，true 是以 event 的形式写进入的，以
event 写进入时，会出现乱码，默认是 true
a0.channels.c1.parseAsFlumeEvent = false
a0.sources.r1.channels = c1
```

这样只要 logs.txt 文件发生任何变化，都是实时的输入到 kafka 的 topic 之中，可以通过以下命令查看。

```
kafka-console-consumer --bootstrap-server ds-bigdata-002:9092,ds-
bigdata-003:9092,ds-bigdata-004:9092 --topic test-wang --from-
beginning
```

4.5.3 企业面试题

1、如何实现 Flume 数据传输的监控

使用第三方框架 Ganglia 实时监控 Flume。监控到 Flume 尝试提交的次数远远大于最终成功的次数，说明 Flume 运行比较差。主要是内存不够导致的。

2、Flume 参数调优

1) source:

batchSize: source 一次批量运输到 Channel 的 event 条数，适当调大这个参数可以提高 Source 搬运 Event 到 Channel 时的性能。

2) channel

type: 选择 memory 时 channel 的性能最好，但是如果 Flume 进程意外挂掉可能会丢失数据。选择 file 时 channel 的容错性更好，但是性能上会比 memory channel 差。

capacity: channel 可容纳最大的 event 条数

transactionCapacity: 每次 Source 往 channel 里面写的最大 event 条数和每次 Sink 从 channel 里面读的最大 event 条数

3) sink

batchSize: Sink 一次批量从 Channel 读取的 event 条数。适当调大这个参数可以提高 Sink 从 Channel 搬出 event 的性能。

3、Flume 的事务机制

Flume 使用**两个独立的事务**分别负责从 Source 到 Channel, 从 Channel 到 Sink 的事件传递。如 spooling directory source 为文件的每一行创建一个事件, 一旦事务中所有的事件全部传递到 Channel 且提交成功, 那么 Source 就将该文件标记为完成。

同理, 事务以类似的方式处理从 Channel 到 Sink 的传递过程, 如果因为某种原因使得事件无法记录, 那么事务将会回滚。且所有的事件都会保持到 Channel 中, 等待重新传递。

4、Flume 采集数据会丢失吗?

生产环境中使用 TailDirSource + FileChannel 所有没有出现丢数据的情况。

根据 Flume 的架构原理, **Flume 本身是不可能丢失数据的**, 其内部有完善的事务机制, Source 到 Channel 是事务性的, Channel 到 Sink 是事务性的, 因此这两个环节不会出现数据的丢失。

可能丢失数据的情况是 Channel 采用 memoryChannel, agent 宕机导致数据丢失, 或者 Channel 存储数据已满, 导致 Source 不再写入, 未写入的数据丢失。

Flume 不会丢失数据, 但是有可能造成数据的重复, 例如数据已经成功由 Sink 发出, 但是没有接收到响应, Sink 会再次发送数据, 此时可能会导致数据的重复。

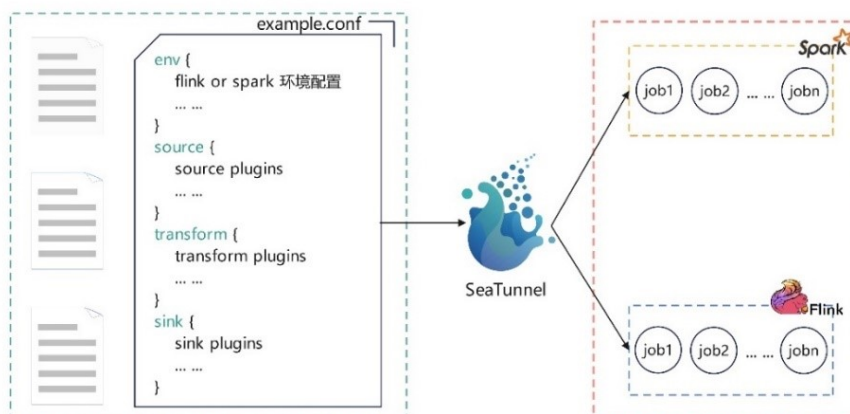
4.6 综合同步解决方案-SeaTunnel☆☆

SeaTunnel 是一个简单易用的数据集成框架, 在企业中, 由于开发时间或开发部门不通用, 往往有多个异构的、运行在不同的软硬件平台上的信息系统同时运行。数据集成是把不同来源、格式、特点性质的数据在逻辑上或物理上有机地集中, 从而为企业提供全面的数据共享。**SeaTunnel 支持海量数据的离线 (BATCH) 与实时 (STREAMING) 同步。**它每天可以稳定高效地同步数百亿数据。并已用于近 100 家公司的生产。

SeaTunnel 可以选择自研的 SeaTunnel Zeta 引擎上运行, 也可以选择在 Apache Flink 或 Spark 引擎上运行。Seaunnel 为实时(CDC)和批量数据提供高性能数据同步能力。本质上, SeaTunnel 不是对 Spark 和 Flink 的内部修改, 而是在 Spark 和 Flink 的基础上做了一层包装。它主要运用了**控制反转**的设计模式, 这也是 SeaTunnel 实现的基本思想。

SeaTunnel 的日常使用, 就是编辑配置文件。编辑好的配置文件由 SeaTunnel 转换为

具体的 Spark 或 Flink 任务。如图所示。



4.6.1 特点及使用场景

丰富且可扩展的连接器：SeaTunnel 提供了一个不依赖于特定执行引擎的连接器 API。基于此 API 开发的连接器（源 Source、转换 Transform、接收器 Sink）可以在许多不同的引擎上运行，例如当前支持的 SeaTunnel Zeta 引擎、Flink、Spark。

连接器插件：插件设计允许用户轻松开发自己的连接器并将其集成到 SeaTunnel 项目中。目前，SeaTunnel 已经支持 100 多个连接器，而且数量还在激增。

批流集成：基于 SeaTunnel 连接器 API 开发的连接器，完美兼容离线同步、实时同步、全同步、增量同步等场景。它大大降低了管理数据集成任务的难度。

多引擎支持：SeaTunnel 默认使用 SeaTunnel 引擎进行数据同步。同时，SeaTunnel 还支持使用 Flink 或 Spark 作为连接器的执行引擎，以适应企业现有的技术组件。SeaTunnel 支持多个版本的 Spark 和 Flink。

JDBC 多路复用，数据库日志多表解析：SeaTunnel 支持多表或全数据库同步，解决了 JDBC 连接过多的问题；支持多表或全库日志读写解析，解决了 CDC 多表同步场景重复读取解析日志的问题。

高吞吐、低时延：SeaTunnel 支持并行读写，提供稳定可靠的数据同步能力，高吞吐、低时延。

完善的实时监控：SeaTunnel 支持数据同步过程中每个步骤的详细监控信息，让用户轻松了解同步任务读写的数据数量、数据大小、QPS 等信息。

适用场景：

涤生大数据

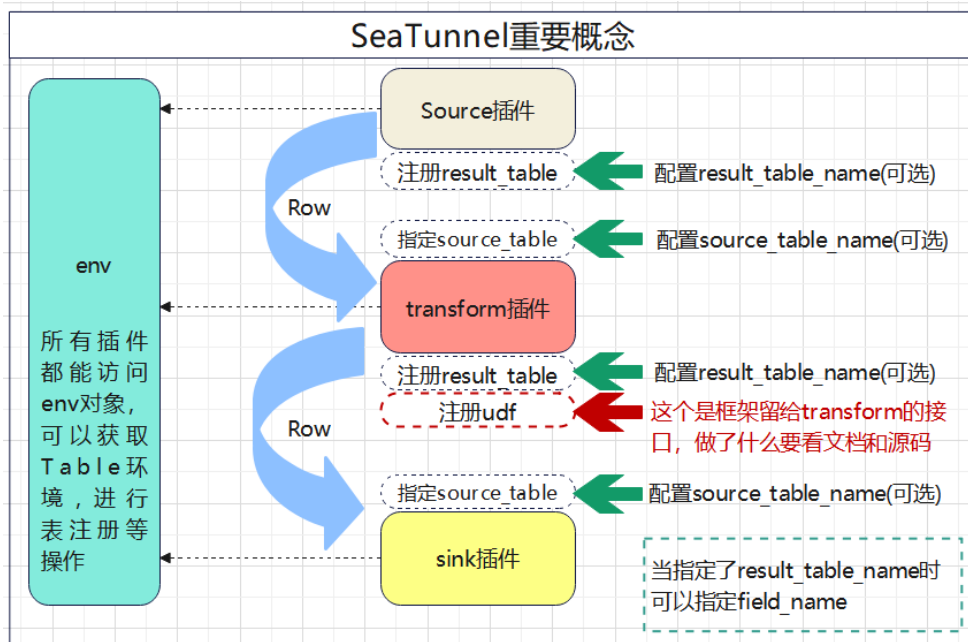
SeaTunnel 适用于以下场景	SeaTunnel 的特点
● 海量数据的同步 ● 海量数据的集成 ● 海量数据的 ETL ● 海量数据聚合 ● 多源数据处理	● 基于配置的低代码开发，易用性高，方便维护。 ● 支持实时流式传输 ● 离线多源数据分析 ● 高性能、海量数据处理能力 ● 模块化的插件架构，易于扩展 ● 支持用 SQL 进行数据操作和数据聚合 ● 支持 Spark structured streaming ● 支持 Spark 2.x

目前 SeaTunnel 的长板是它有丰富的连接器，又因为它以 Spark 和 Flink 为引擎。所以可以很好地进行分布式的海量数据同步。**通常 SeaTunnel 会被用来做出仓入仓工具，或者被用来进行数据集成。**比如，唯品会就选择用 SeaTunnel 来解决数据孤岛问题，让 ClickHouse 集成到了企业中先前的数据系统之中。

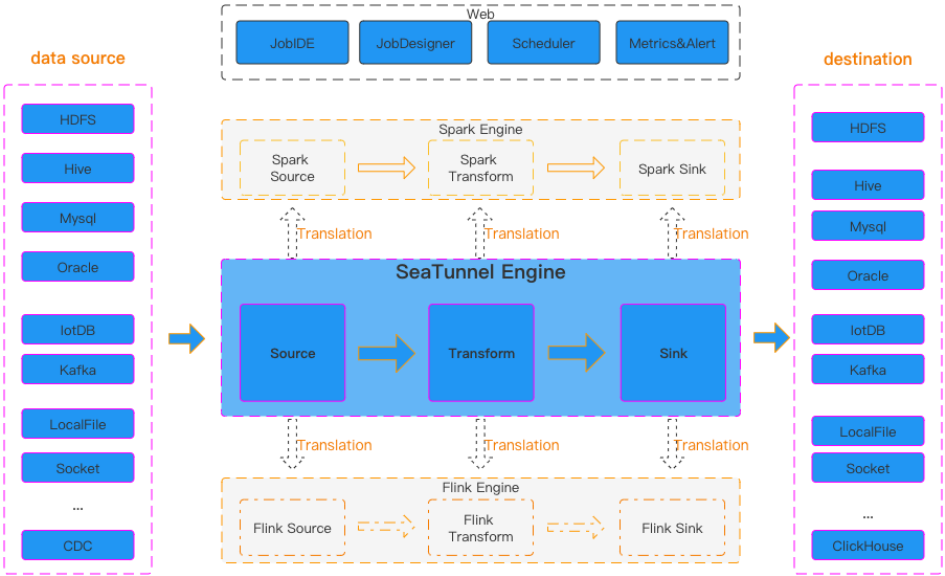
4.6.2 架构设计

SeaTunnel 设计的核心是利用设计模式中的“控制翻转”或者叫“依赖注入”，主要概括为以下两点：

- 1、上层不依赖底层，两者都依赖抽象；
- 2、流程代码与业务逻辑应该分离。整个数据处理过程，大致可以分为以下几个流程：**输入 (Source) -> 转换 (Transform) -> 输出 (Sink)**，对于更复杂的数据处理，实质上也是这几种行为的组合：



下图是 SeaTunnel 的工作流程：



4.6.3 执行引擎类型及支持的数据源

Spark 连接器插件(Source):

Spark 连接器插件	数据库类型	Source	Sink
Batch	Fake	✓	
	ElasticSearch	✓	✓
	File	✓	✓
	Hive	✓	✓
	Hudi	✓	✓
	Jdbc	✓	✓
	MongoDB	✓	✓
	Neo4j	✓	
	Phoenix	✓	✓
	Redis	✓	✓
	Tidb	✓	✓
	Clickhouse		✓
	Doris		✓
	Email		✓
	Hbase	✓	✓
	Kafka		✓
	Console		✓
Stream	Kudu	✓	✓
	Redis	✓	✓
	FakeStream	✓	
	KafkaStream	✓	
	SocketStream	✓	

Flink 连接器插件(Source):

Flink 连接器插件	数据库类型	Source	Sink
	Druid	✓	✓
	Fake	✓	
	File	✓	✓
	InfluxDb	✓	✓
	Jdbc	✓	✓
	Kafka	✓	✓
	Socket	✓	
	Console		✓
	Doris		✓
	ElasticSearch		✓

这部分从官网可以看出社区目前规划了大量的插件，但截至 V2.3.3 可用的 transform 插件的数量还是很少的，比如只有 split、sql、fieldmapper 等。同学们有兴趣也可以在业余时间尝试参与开源贡献。官方网址：<https://seatunnel.apache.org/zh-CN/>

4.6.4 安装与使用

前置条件：

目前使用的 seatunnel 的版本是最新发布的 2.3.3 的版本，使用 spark 的版本也是比较新的 3.x 版本，使用的 flink 的版本也是相对较新的 1.15，java 还是使用最稳定的 1.8.

SeaTunnel 的下载与安装

- 1) 使用 wget 下载 SeaTunnel

```
get https://dlcdn.apache.org/seatunnel/2.3.3/apache-seatunnel-2.3.3-bin.tar.gz
```

- 2) 解压下载好的 tar.gz 包

```
tar -zxvf apache-seatunnel-2.3.3-bin.tar.gz
```

- 3) seatunnel 的依赖环境配置

在 config/目录中有一个 seatunnel-env.sh 脚本，可以看一下里面的内容。

```
# Home directory of spark distribution.
SPARK_HOME=${SPARK_HOME:-/opt/spark}
# Home directory of flink distribution.
FLINK_HOME=/opt/cloudera/parcels/FLINK/lib/flink
```

这个脚本中声明了 SPARK_HOME 和 FLINK_HOME 两个路径。默认情况下 seatunnel-env.sh 中的 SPARK_HOME 和 FLINK_HOME 就是系统环境变量中的 SPARK_HOME 和 FLINK_HOME。

然后我们再来看 seatunnel/bin 目录下的一些启动文件：


```
[root@ds-bigdata-005 bin]# ls -al
total 40
drwxr-xr-x  2 root root 4096 Feb 18  2022 .
drwxr-xr-x 11 root root 4096 Nov 22 21:37 ..
-rwxr-xr-x  1 root root 2328 Feb 18  2022 install-plugin.sh
-rwxr-xr-x  1 root root 3381 Feb 18  2022 seatunnel-cluster.sh
-rwxr-xr-x  1 root root 3119 Feb 18  2022 seatunnel.sh
-rwxr-xr-x  1 root root 2220 Feb 18  2022 start-seatunnel-flink-13-connector-v2.sh
-rwxr-xr-x  1 root root 2220 Feb 18  2022 start-seatunnel-flink-15-connector-v2.sh
-rwxr-xr-x  1 root root 2219 Feb 18  2022 start-seatunnel-spark-2-connector-v2.sh
-rwxr-xr-x  1 root root 2219 Feb 18  2022 start-seatunnel-spark-3-connector-v2.sh
-rwxr-xr-x  1 root root 2066 Feb 18  2022 stop-seatunnel-cluster.sh
```

install-plugin.sh	--安装连接器脚本
seatunnel-cluster.sh	--集群模式启动脚本
seatunnel.sh	--本地模式启动脚本
start-seatunnel-flink-13-connector-v2.sh	--flink1.13 版本引擎启动脚本
start-seatunnel-flink-15-connector-v2.sh	--flink1.15 版本引擎启动脚本
start-seatunnel-spark-2-connector-v2.sh	--spark 版本引擎启动脚本
start-seatunnel-spark-3-connector-v2.sh	--spark 版本引擎启动脚本
stop-seatunnel-cluster.sh	--集群模式停止脚本

另外从 seatunnel v2.2.0 开始，二进制包默认不提供连接器依赖，所以第一次使用时，我们需要下载连接器。在 \${seatunnel_home}/config/plugin_config 文件中指定下载哪些连接器，可以修改这个配置指定下载对应的连接器，连接器会下载到这个目录下 \${seatunnel_home}/connectors/seatunnel/ 中。所以第一步需要下载连接器

```
# 备份原有的组件配置
cp ${SEATUNNEL_HOME}/config/plugin_config
${SEATUNNEL_HOME}/config/plugin_config_bak

# 创建一个新的
vi ${SEATUNNEL_HOME}/config/plugin_config

# 只安装这三种插件，因为他的插件太多了，下载很慢，用哪个下哪个
--connectors-v2--
connector-jdbc
connector-kafka
connector-console
--end--

# 运行安装命令
sh ${SEATUNNEL_HOME}/bin/install-plugin.sh 2.3.3
```

```
[WARNING] Notice transitive dependencies won't be copied.
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 0.778 s
[INFO] Finished at: 2023-11-22T22:47:57+08:00
[INFO] -----
install connector : connector-selectdb-cloud
[INFO] Scanning for projects ...
[INFO] -----< org.apache.maven:standalone-pom >-----
[INFO] Building Maven Stub Project (No POM) 1
[INFO] -----[ pom ]-----
[INFO] --- maven-dependency-plugin:2.8:get (default-cli) @ standalone-pom ---
[WARNING] The artifact xml-apis:xml-apis:jar:2.0.2 has been relocated to xml-apis:xml-apis:jar:1.0.b2
[INFO] Resolving org.apache.seatunnel:connector-selectdb-cloud:jar:2.3.3 with transitive dependencies
[WARNING] destination/dest parameter is deprecated: it will disappear in future version.
[INFO] Copying /root/.m2/repository/org/apache/seatunnel/connector-selectdb-cloud/2.3.3/connector-selectdb-cloud-2.3.3.jar to
seatunnel
[WARNING] Notice transitive dependencies won't be copied.
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 0.788 s
[INFO] Finished at: 2023-11-22T22:47:58+08:00
[INFO] -----
install connector : connector-hbase
```

然后还要在 lib 目录下添加资源 jar 包

```
cd ${SEATUNNEL_HOME}/lib
# 添加以下依赖 jar, 连接 mysql 需要使用
mysql-connector-j-8.0.31.jar
```

4.6.5 案例 1: 入门项目

1、编写任务 demo.conf

```
env {
  execution.parallelism = 1
  job.mode = "BATCH"
}
source {
  FakeSource {
    result_table_name = "fake"
    row.num = 16
    schema = {
      fields {
        name = "string"
        age = "int"
      }
    }
  }
}
```

涤生大数据

```
transform {  
  
}  
  
sink {  
  Console {}  
}
```

2. 使用本地模式运行该程序并查看结果

```
./seatunnel.sh --config ../config/demo.conf -e local
```

```
2023-11-22 22:05:30.837 INFO org.apache.seatunnel.engine.server.task.SourceSplitEnumeratorTask - received enough reader, starting enumerator ...  
2023-11-22 22:05:30.837 INFO org.apache.seatunnel.connectors.seatunnel.fake.source.FakeSourceSplitEnumerator - Starting to calculate splits.  
2023-11-22 22:05:30.838 INFO org.apache.seatunnel.connectors.seatunnel.fake.source.FakeSourceSplitEnumerator - Calculated splits successfully, the size of splits is 1.  
2023-11-22 22:05:30.838 INFO org.apache.seatunnel.connectors.seatunnel.fake.source.FakeSourceSplitEnumerator - Assigning splits to readers 0 [fakeSourceSplit(splitId=0, rowNum=16)] to 1 readers.  
2023-11-22 22:05:30.845 INFO org.apache.seatunnel.connectors.seatunnel.console.sink.ConsoleSinkWriter - 16 rows of data have been generated in split(0). Generation time: 170061938843  
2023-11-22 22:05:30.846 INFO org.apache.seatunnel.connectors.seatunnel.console.sink.ConsoleSinkWriter - subtaskIndex=0 row=0: SeaTunnelRowKind=INSERT : SFWu, 248152768  
2023-11-22 22:05:30.846 INFO org.apache.seatunnel.connectors.seatunnel.console.sink.ConsoleSinkWriter - subtaskIndex=0 row=1: SeaTunnelRowKind=INSERT : swhG, 172716864  
2023-11-22 22:05:30.846 INFO org.apache.seatunnel.connectors.seatunnel.console.sink.ConsoleSinkWriter - subtaskIndex=0 row=2: SeaTunnelRowKind=INSERT : RWbLv, 135499208  
2023-11-22 22:05:30.846 INFO org.apache.seatunnel.connectors.seatunnel.console.sink.ConsoleSinkWriter - subtaskIndex=0 row=3: SeaTunnelRowKind=INSERT : LpSz, 66333368  
2023-11-22 22:05:30.846 INFO org.apache.seatunnel.connectors.seatunnel.console.sink.ConsoleSinkWriter - subtaskIndex=0 row=4: SeaTunnelRowKind=INSERT : HqMa, 117392895  
2023-11-22 22:05:30.846 INFO org.apache.seatunnel.connectors.seatunnel.console.sink.ConsoleSinkWriter - subtaskIndex=0 row=5: SeaTunnelRowKind=INSERT : cLpP, 15513117  
2023-11-22 22:05:30.846 INFO org.apache.seatunnel.connectors.seatunnel.console.sink.ConsoleSinkWriter - subtaskIndex=0 row=6: SeaTunnelRowKind=INSERT : cYfQ, 167948292  
2023-11-22 22:05:30.846 INFO org.apache.seatunnel.connectors.seatunnel.console.sink.ConsoleSinkWriter - subtaskIndex=0 row=7: SeaTunnelRowKind=INSERT : AOtU, 47497128  
2023-11-22 22:05:30.846 INFO org.apache.seatunnel.connectors.seatunnel.console.sink.ConsoleSinkWriter - subtaskIndex=0 row=8: SeaTunnelRowKind=INSERT : aRoje, 109341678  
2023-11-22 22:05:30.846 INFO org.apache.seatunnel.connectors.seatunnel.console.sink.ConsoleSinkWriter - subtaskIndex=0 row=9: SeaTunnelRowKind=INSERT : tMEq, 124326888  
2023-11-22 22:05:30.846 INFO org.apache.seatunnel.connectors.seatunnel.console.sink.ConsoleSinkWriter - subtaskIndex=0 row=10: SeaTunnelRowKind=INSERT : yDhW, 51895024  
2023-11-22 22:05:30.846 INFO org.apache.seatunnel.connectors.seatunnel.console.sink.ConsoleSinkWriter - subtaskIndex=0 row=11: SeaTunnelRowKind=INSERT : IDyH, 128897868  
2023-11-22 22:05:30.846 INFO org.apache.seatunnel.connectors.seatunnel.console.sink.ConsoleSinkWriter - subtaskIndex=0 row=12: SeaTunnelRowKind=INSERT : kcdM, 58668236  
2023-11-22 22:05:30.846 INFO org.apache.seatunnel.connectors.seatunnel.console.sink.ConsoleSinkWriter - subtaskIndex=0 row=13: SeaTunnelRowKind=INSERT : SuLGo, 68217128  
2023-11-22 22:05:30.846 INFO org.apache.seatunnel.connectors.seatunnel.console.sink.ConsoleSinkWriter - subtaskIndex=0 row=14: SeaTunnelRowKind=INSERT : qYfX, 126388736  
2023-11-22 22:05:30.846 INFO org.apache.seatunnel.connectors.seatunnel.console.sink.ConsoleSinkWriter - subtaskIndex=0 row=15: SeaTunnelRowKind=INSERT : jYwS, 837571897
```

3. 使用 spark 引擎运行该程序并查看结果

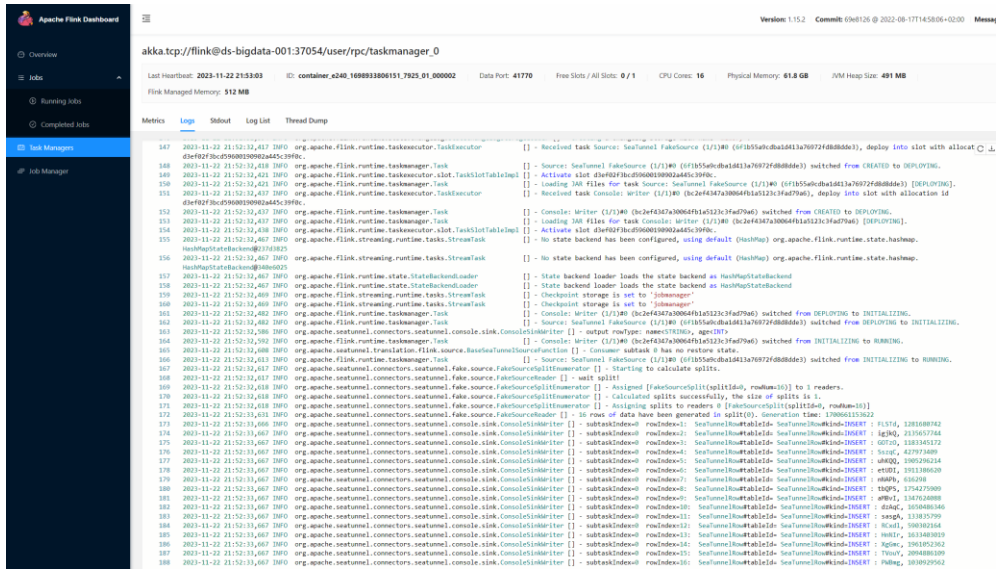
```
./start-seatunnel-spark-3-connector-v2.sh --master local[4] --deploy-mode client --config ../config/demo.conf
```

```
artitions Vector(0))  
23/11/22 21:53:30 INFO TaskSchedulerImpl: Adding task set 0.0 with 1 tasks resource profile 0  
23/11/22 21:53:30 INFO TaskSetManager: Starting task 0.0 in stage 0.0 (TID 0) (ds-bigdata-005, executor driver, partition 0, PROCESS_LOCAL, ents Map())  
23/11/22 21:53:30 INFO Executor: Running task 0.0 in stage 0.0 (TID 0)  
23/11/22 21:53:31 INFO ConsoleSinkWriter: output rowType: name<STRING>, age<INT>  
23/11/22 21:53:31 INFO FakeSourceReader: wait split!  
23/11/22 21:53:31 INFO FakeSourceSplitEnumerator: Starting to calculate splits.  
23/11/22 21:53:31 INFO FakeSourceSplitEnumerator: Calculated splits successfully, the size of splits is 1.  
23/11/22 21:53:31 INFO FakeSourceSplitEnumerator: Assigning splits to readers 0 [FakeSourceSplit(splitId=0, rowNum=16)]  
23/11/22 21:53:31 INFO FakeSourceReader: 16 rows of data have been generated in split(0). Generation time: 1700661212170  
23/11/22 21:53:32 INFO ConsoleSinkWriter: subtaskIndex=0 row=0: SeaTunnelRowKind=INSERT : K0LfH, 1387864292  
23/11/22 21:53:32 INFO ConsoleSinkWriter: subtaskIndex=0 row=1: SeaTunnelRowKind=INSERT : PTEmO, 822903289  
23/11/22 21:53:32 INFO ConsoleSinkWriter: subtaskIndex=0 row=2: SeaTunnelRowKind=INSERT : VtLYw, 2029136092  
23/11/22 21:53:32 INFO ConsoleSinkWriter: subtaskIndex=0 row=3: SeaTunnelRowKind=INSERT : LDxfX, 965313848  
23/11/22 21:53:32 INFO ConsoleSinkWriter: subtaskIndex=0 row=4: SeaTunnelRowKind=INSERT : bDzmZ, 1483833378  
23/11/22 21:53:32 INFO ConsoleSinkWriter: subtaskIndex=0 row=5: SeaTunnelRowKind=INSERT : qVvVK, 1547176377  
23/11/22 21:53:32 INFO ConsoleSinkWriter: subtaskIndex=0 row=6: SeaTunnelRowKind=INSERT : QVtTo, 1963249712  
23/11/22 21:53:32 INFO ConsoleSinkWriter: subtaskIndex=0 row=7: SeaTunnelRowKind=INSERT : jNaGt, 850515031  
23/11/22 21:53:32 INFO ConsoleSinkWriter: subtaskIndex=0 row=8: SeaTunnelRowKind=INSERT : S0xdC, 35379503  
23/11/22 21:53:32 INFO ConsoleSinkWriter: subtaskIndex=0 row=9: SeaTunnelRowKind=INSERT : mtLds, 1663869052  
23/11/22 21:53:32 INFO ConsoleSinkWriter: subtaskIndex=0 row=10: SeaTunnelRowKind=INSERT : zeDyM, 1493697431  
23/11/22 21:53:32 INFO ConsoleSinkWriter: subtaskIndex=0 row=11: SeaTunnelRowKind=INSERT : MrFXf, 688959468  
23/11/22 21:53:32 INFO ConsoleSinkWriter: subtaskIndex=0 row=12: SeaTunnelRowKind=INSERT : DDtwm, 667272887  
23/11/22 21:53:32 INFO ConsoleSinkWriter: subtaskIndex=0 row=13: SeaTunnelRowKind=INSERT : qmbPk, 1508219970  
23/11/22 21:53:32 INFO ConsoleSinkWriter: subtaskIndex=0 row=14: SeaTunnelRowKind=INSERT : AKhDP, 686694732  
23/11/22 21:53:32 INFO ConsoleSinkWriter: subtaskIndex=0 row=15: SeaTunnelRowKind=INSERT : CLJfJ, 1440819316
```

4. 使用 flink 引擎运行该程序并查看结果

```
./start-seatunnel-flink-15-connector-v2.sh -m yarn-cluster -ynm  
seatunnel-first --config ../config/demo.conf
```

涤生大数据



Apache Flink Dashboard

akka.tcp://flink@ds-bigdata-001:37054/user/rpc/taskmanager_0

Last heartbeat: 2023-11-22 21:53:03 ID: container_a240_1408931806151_7925_01_000002 Data Port: 41770 Free Slots / All Slots: 0 / 1 CPU Cores: 16 Physical Memory: 61.8 GB JVM Heap Size: 491 MB Flink Managed Memory: 512 MB

Metrics Logs Status Log List Thread Dump

147 2023-11-22 21:52:12,417 INFO org.apache.flink.runtime.taskexecutor.TaskExecutor [] - Received task Source: Seafunnel FakeSource (1/1)80 (0f18554bcbda1d1a76972f4d88bde3), deploy into slot with allocation Id b2e4f347a30064f1a5123c3fa796a.

148 2023-11-22 21:52:12,418 INFO org.apache.flink.runtime.taskmanager.Task [] - Source: Seafunnel FakeSource (1/1)80 (0f18554bcbda1d1a76972f4d88bde3) switched from CREATED to DEPLOYING.

149 2023-11-22 21:52:12,421 INFO org.apache.flink.runtime.taskmanager.slot.TaskSlotTableImpl [] - Activate slot b2e4f347a30064f1a5123c3fa796a.

150 2023-11-22 21:52:12,423 INFO org.apache.flink.runtime.taskmanager.Task [] - Loading 100 files for task Source: Seafunnel FakeSource (1/1)80 (0f18554bcbda1d1a76972f4d88bde3) [DEPLOYING].

151 2023-11-22 21:52:12,427 INFO org.apache.flink.runtime.taskexecutor.TaskExecutor [] - Received task Console: Writer (1/1)80 (b2e4f347a30064f1a5123c3fa796a), deploy into slot with allocation Id b2e4f347a30064f1a5123c3fa796a.

152 2023-11-22 21:52:12,437 INFO org.apache.flink.runtime.taskmanager.Task [] - Console: Writer (1/1)80 (b2e4f347a30064f1a5123c3fa796a) switched from CREATED to DEPLOYING.

153 2023-11-22 21:52:12,437 INFO org.apache.flink.runtime.taskmanager.Task [] - Loading 100 files for task Console: Writer (1/1)80 (b2e4f347a30064f1a5123c3fa796a) [DEPLOYING].

154 2023-11-22 21:52:12,438 INFO org.apache.flink.runtime.taskmanager.slot.TaskSlotTableImpl [] - Activate slot b2e4f347a30064f1a5123c3fa796a.

155 2023-11-22 21:52:12,467 INFO org.apache.flink.streaming.runtime.tasks.StreamTask [] - No state backend has been configured, using default (HashMap) org.apache.flink.runtime.state.hashmap.HashMapStateBackend@1103025.

156 2023-11-22 21:52:12,467 INFO org.apache.flink.streaming.runtime.tasks.StreamTask [] - No state backend has been configured, using default (HashMap) org.apache.flink.runtime.state.hashmap.HashMapStateBackend@1103025.

157 2023-11-22 21:52:12,467 INFO org.apache.flink.runtime.state.StateBackendLoader [] - State backend loader loads the state backend as HashMapStateBackend

158 2023-11-22 21:52:12,467 INFO org.apache.flink.runtime.state.StateBackendLoader [] - State backend loader loads the state backend as HashMapStateBackend

159 2023-11-22 21:52:12,469 INFO org.apache.flink.streaming.runtime.tasks.StreamTask [] - Checkpoint storage is set to 'jobmanager'

160 2023-11-22 21:52:12,469 INFO org.apache.flink.streaming.runtime.tasks.StreamTask [] - Checkpoint storage is set to 'jobmanager'

161 2023-11-22 21:52:12,482 INFO org.apache.flink.runtime.taskmanager.Task [] - Console: Writer (1/1)80 (b2e4f347a30064f1a5123c3fa796a) switched from DEPLOYING to INITIALIZING.

162 2023-11-22 21:52:12,482 INFO org.apache.flink.runtime.taskmanager.Task [] - Source: Seafunnel FakeSource (1/1)80 (0f18554bcbda1d1a76972f4d88bde3) switched from DEPLOYING to INITIALIZING.

163 2023-11-22 21:52:12,486 INFO org.apache.flink.runtime.taskmanager.Task [] - Console: Writer (1/1)80 (b2e4f347a30064f1a5123c3fa796a) switched from INITIALIZING to RUNNING.

164 2023-11-22 21:52:12,486 INFO org.apache.flink.runtime.taskmanager.Task [] - Console: Writer (1/1)80 (b2e4f347a30064f1a5123c3fa796a) switched from INITIALIZING to RUNNING.

165 2023-11-22 21:52:12,488 INFO org.apache.seatunnel.translation.flink.source.SeaTunnelSourceFunction [] - Consumer subtask 0 has no restore state.

166 2023-11-22 21:52:12,491 INFO org.apache.flink.runtime.taskmanager.Task [] - Source: Seafunnel FakeSource (1/1)80 (0f18554bcbda1d1a76972f4d88bde3) switched from INITIALIZING to RUNNING.

167 2023-11-22 21:52:12,491 INFO org.apache.seatunnel.connectors.seatunnel.fake.source.FakeSourceSplitGenerator [] - Starting to calculate splits.

168 2023-11-22 21:52:12,491 INFO org.apache.seatunnel.connectors.seatunnel.fake.source.FakeSourceSplitGenerator [] - split split1

169 2023-11-22 21:52:12,491 INFO org.apache.seatunnel.connectors.seatunnel.fake.source.FakeSourceSplitGenerator [] - Assigned [FakeSourceSplitGenerator] to 1 readers.

170 2023-11-22 21:52:12,491 INFO org.apache.seatunnel.connectors.seatunnel.fake.source.FakeSourceSplitGenerator [] - Calculated splits successfully, the size of split is 1.

171 2023-11-22 21:52:12,491 INFO org.apache.seatunnel.connectors.seatunnel.fake.source.FakeSourceSplitGenerator [] - Assigning splits to readers 0 [FakeSourceSplitGenerator, reader=0]

172 2023-11-22 21:52:12,491 INFO org.apache.seatunnel.connectors.seatunnel.fake.source.FakeSourceSplitGenerator [] - 10 rows of data have been generated in split(0). Generation time: 170662113022

173 2023-11-22 21:52:12,491 INFO org.apache.seatunnel.connectors.seatunnel.console.sink.ConsoleSinkWriter [] - subtaskIndex=0 readerIndex=0 SeafunnelRowTableId: SeafunnelRowTableId-100001 - 1ST, 1300000002

174 2023-11-22 21:52:12,491 INFO org.apache.seatunnel.connectors.seatunnel.console.sink.ConsoleSinkWriter [] - subtaskIndex=0 readerIndex=0 SeafunnelRowTableId: SeafunnelRowTableId-100001 - 1ST, 1300000002

175 2023-11-22 21:52:12,491 INFO org.apache.seatunnel.connectors.seatunnel.console.sink.ConsoleSinkWriter [] - subtaskIndex=0 readerIndex=0 SeafunnelRowTableId: SeafunnelRowTableId-100001 - 1ST, 1300000002

176 2023-11-22 21:52:12,491 INFO org.apache.seatunnel.connectors.seatunnel.console.sink.ConsoleSinkWriter [] - subtaskIndex=0 readerIndex=0 SeafunnelRowTableId: SeafunnelRowTableId-100001 - 1ST, 1300000002

177 2023-11-22 21:52:12,491 INFO org.apache.seatunnel.connectors.seatunnel.console.sink.ConsoleSinkWriter [] - subtaskIndex=0 readerIndex=0 SeafunnelRowTableId: SeafunnelRowTableId-100001 - 1ST, 1300000002

178 2023-11-22 21:52:12,491 INFO org.apache.seatunnel.connectors.seatunnel.console.sink.ConsoleSinkWriter [] - subtaskIndex=0 readerIndex=0 SeafunnelRowTableId: SeafunnelRowTableId-100001 - 1ST, 1300000002

179 2023-11-22 21:52:12,491 INFO org.apache.seatunnel.connectors.seatunnel.console.sink.ConsoleSinkWriter [] - subtaskIndex=0 readerIndex=0 SeafunnelRowTableId: SeafunnelRowTableId-100001 - 1ST, 1300000002

180 2023-11-22 21:52:12,491 INFO org.apache.seatunnel.connectors.seatunnel.console.sink.ConsoleSinkWriter [] - subtaskIndex=0 readerIndex=0 SeafunnelRowTableId: SeafunnelRowTableId-100001 - 1ST, 1300000002

181 2023-11-22 21:52:12,491 INFO org.apache.seatunnel.connectors.seatunnel.console.sink.ConsoleSinkWriter [] - subtaskIndex=0 readerIndex=0 SeafunnelRowTableId: SeafunnelRowTableId-100001 - 1ST, 1300000002

182 2023-11-22 21:52:12,491 INFO org.apache.seatunnel.connectors.seatunnel.console.sink.ConsoleSinkWriter [] - subtaskIndex=0 readerIndex=0 SeafunnelRowTableId: SeafunnelRowTableId-100001 - 1ST, 1300000002

183 2023-11-22 21:52:12,491 INFO org.apache.seatunnel.connectors.seatunnel.console.sink.ConsoleSinkWriter [] - subtaskIndex=0 readerIndex=0 SeafunnelRowTableId: SeafunnelRowTableId-100001 - 1ST, 1300000002

184 2023-11-22 21:52:12,491 INFO org.apache.seatunnel.connectors.seatunnel.console.sink.ConsoleSinkWriter [] - subtaskIndex=0 readerIndex=0 SeafunnelRowTableId: SeafunnelRowTableId-100001 - 1ST, 1300000002

185 2023-11-22 21:52:12,491 INFO org.apache.seatunnel.connectors.seatunnel.console.sink.ConsoleSinkWriter [] - subtaskIndex=0 readerIndex=0 SeafunnelRowTableId: SeafunnelRowTableId-100001 - 1ST, 1300000002

186 2023-11-22 21:52:12,491 INFO org.apache.seatunnel.connectors.seatunnel.console.sink.ConsoleSinkWriter [] - subtaskIndex=0 readerIndex=0 SeafunnelRowTableId: SeafunnelRowTableId-100001 - 1ST, 1300000002

187 2023-11-22 21:52:12,491 INFO org.apache.seatunnel.connectors.seatunnel.console.sink.ConsoleSinkWriter [] - subtaskIndex=0 readerIndex=0 SeafunnelRowTableId: SeafunnelRowTableId-100001 - 1ST, 1300000002

188 2023-11-22 21:52:12,491 INFO org.apache.seatunnel.connectors.seatunnel.console.sink.ConsoleSinkWriter [] - subtaskIndex=0 readerIndex=0 SeafunnelRowTableId: SeafunnelRowTableId-100001 - 1ST, 1300000002

备注：如果指定 `job.mode = "BATCH"`, flink 执行完之后就会立即结束，需要将 `job.mode = "STREAMING"`之后，才可以是常驻任务。

4.6.5 案例 2：transform Split

1、编写任务 split.conf

```
env {
    execution.parallelism = 1
    job.mode = "STREAMING"
}

source {
    Socket {
        host = "ds-bigdata-005"
        port = 9990
        result_table_name="fake"
        field_name="info"
    }
}

transform {
    Split {
        source_table_name = "fake"
    }
}
```

```
result_table_name = "fake1"
separator = "#"
split_field = "info"
output_fields = [first_name, second_name]
}
}

sink {
  Console {
    source_table_name = "fake1"
  }
}
```

因为以上的任务是需要监控要给的端口，因此可以使用 nc 先启动一个固定端口

```
nc -l 9990
```

使用 flink 引擎运行该程序并查看结果

```
./start-seatunnel-flink-15-connector-v2.sh -m yarn-cluster -
ynm seatunnel-first --config ../config/split.conf
```

```
2023-11-22 15:41:19,876 INFO org.apache.seatunnel.translation.flink.source.BaseFlinkSourceFunction [ ] - Consumer subtask has no restore state
2023-11-22 15:41:19,880 INFO org.apache.seatunnel.connector.socket.SocketSource [ ] - Socket source, host[id=SinkConnector] port[port90]
2023-11-22 15:41:19,881 INFO org.apache.flink.runtime.taskmanager.Task [ ] - Source: SeatunnelSocketSource> SourceConnector[1]> SinkConnector[2]> Flat Map > FakeId > SinkConnector[4]
2023-11-22 15:41:19,881 INFO org.apache.seatunnel.connector.console.sink.ConsoleSinkWriter [ ] - subtaskIndex=rowIndex: SeatunnelRowIdField->SeatunnelRowIdField->nihaochangpin, nihaochangpin, nihaochangpin
2023-11-22 15:41,883 INFO org.apache.seatunnel.connectors.seatunnel.console.sink.ConsoleSinkWriter [ ] - subtaskIndex=rowIndex: SeatunnelRowIdField->SeatunnelRowIdField-INJECT uohaojinjin, uohao, jinjin
2023-11-22 15:41,886 INFO org.apache.seatunnel.connectors.seatunnel.console.sink.ConsoleSinkWriter [ ] - subtaskIndex=rowIndex: SeatunnelRowIdField->SeatunnelRowIdField-INJECT changangfeng, changang, jian
2023-11-22 15:41,888 INFO org.apache.seatunnel.connectors.seatunnel.console.sink.ConsoleSinkWriter [ ] - subtaskIndex=rowIndex: SeatunnelRowIdField->SeatunnelRowIdField-INJECT wangqinling, wangqinling, wangqinling
2023-11-22 15:41,891 INFO org.apache.seatunnel.connectors.seatunnel.console.sink.ConsoleSinkWriter [ ] - subtaskIndex=rowIndex: SeatunnelRowIdField->SeatunnelRowIdField-INJECT wangsufanling, wangsufan, flink
2023-11-22 15:41,900 INFO org.apache.seatunnel.connectors.seatunnel.console.sink.ConsoleSinkWriter [ ] - subtaskIndex=rowIndex: SeatunnelRowIdField->SeatunnelRowIdField-INJECT zhangyue, zhangyue, zheliu
2023-11-22 15:41,903 INFO org.apache.seatunnel.connectors.seatunnel.console.sink.ConsoleSinkWriter [ ] - subtaskIndex=rowIndex: SeatunnelRowIdField->SeatunnelRowIdField-INJECT
```

5 数据同步常见的问题

5.1 数据源多、可选组件多的问题☆☆

一般我们说的**数据同步的方法通常的步骤**就是先创建目标表，再通过同步工具的填写数据库连接、表、字段等各种配置信息后测试完成数据同步。

但是这其中有**很多的问题**：

- i、业务发展，数据量增大，传统方法完成工作的工作量增大，而且，相似并重复的操作降低开发人员的工作热情；
- ii、数据仓库的数据源种类特别丰富，遇到不同类型的数据同步，开发人员需要去了解其特色配置；
- iii、市面上可用的组件过多，如何甄别这些组件是否真正的满足需求
- iv、部分真正的数据需求方，如 Java 开发和业务运营，由于存在相关数据同步的专业技能部门，往往需要将需求提交给数据开发方来完成，额外增加了沟通和流程成本；

解决方案以阿里集团为例：数据仓库研发了 OneClick 产品；

在 OneData 体系里，让数据研发更高效的工具有一键同步。OneClick 顾名思义，就是**通过一键点击的方式，来提高数据研发的效率**。目前 OneClick 主要覆盖的场景有两个：一是数据同步，另一个是存量数据日常维护。**OneClick 的思路实际上就是将数据研发一些日常的操作，通过一个可复用的流程，一键点击、一键完成**。在上述两块数据处理的场景中，所有的功能都是基于某一事先配置好任务流去实现，每个任务流都是不同的 API 接口按照一定顺序的排列组合。



通常我们可以这么做：

- i、对不同数据源的数据同步**配置透明化**，可以通过库名和表名唯一定位，通过 IDB (Interface DataBase) 接口获取源数据信息自动生成配合信息；

- ii、简化了数据同步操作步骤，实现了与数据同步相关的建表、配置任务、发布、测试操作一键化处理，并且封装成 Web 接口进一步达到批量化的效果；
- iii、**降低了数据同步的技能门槛，让数据需求方更加方便地获取和使用数据。**

因此 OneClick 实现了数据的一键化和批量化同步，一键完成 DDL 和 DML 的生成、数据的冒烟测试（**冒烟测试**就是完成一个新版本的开发后，对该版本最基本的功能进行测试，保证基本的功能和流程能走通）以及在生成环境中测试等。

备注：IDB：阿里集团用于统一管理 MySQL、OceanBase、PostgreSQL、Oracle、SQL Server 等关系型数据库的平台，是一种集数据管理、结构管理、诊断优化、实时监控和系统管理与一体的数据管理服务。（在对集团数据库表统一管服务过程中，IDB 产出了数据库、表、字段各个级别元数据信息，并提供了元数据接口服务）

5.2 零点漂移的问题☆☆

数据零点漂移指的是数据同步过程中，ODS 表按时间字段分区时，同一个业务日期（分区）包含前一天的数据或丢失了当天的数据、或者包含后一天凌晨附近的数据，**一般是对增量表的情况而言。**

由于 ODS 需要承接面向历史的细节数据查询需求，这就需要物理落地到数据仓库的 ODS 表按时间段来切分进行分区存储，通常的做法是**按某些时间戳字段来分区**，而实际上往往由于**时间戳字段的准确性问题**导致发生数据漂移。

5.2.1 原因分析

当数仓 ODS 采用按时间段分区的方式存储数据，时间字段的选择会导致不同类型的数据漂移现象。通常，时间戳字段分为四类：

数据库表中用来标识数据记录**更新时间**的时间戳字段（modified_time）

数据库日志中用来标识数据记录**更新时间**的时间戳字段（log_time）

数据库表中用来记录具体**业务过程发生时间**的时间戳字段（proc_time）

标识数据记录**被抽取到时间**的时间戳字段（extract_time）

理论上，上述四个时间戳应该是一致的，即 $\text{proc_time} = \text{log_time} = \text{modified_time} = \text{extract_time}$ ，但实际生产中，这几个时间往往会出现差异。在现实中，四个时间戳的大小关系为：

$\text{poc_time} < \text{log_time} < \text{modified_time} < \text{extract_time}$

造成这些差异的原因有：

- 1、数据产生后才能抽数，并且很难做到数据实时产生实时抽取，所以 extract_time 一般会晚于其他三个时间。

2、关系型数据库采用**预写日志方式 (WAL, Write Ahead Log)** 来更新数据, 所以更新时间 modified_time 会晚于 log_time。

3、网络或系统压力问题, 会导致数据延迟写入数据延迟更新。log_time 或者 modified_time 会晚于 proc_time。

根据其中的某一个时间字段来切分 ODS 表, 这就导致产生数据漂移。下面来具体看下数据漂移的**几种场景**:

1、extract_time 抽取时间分区, 这种情况数据漂移的问题最明显。

2、modified_time 更新时间分区。在实际生产中这种情况最常见, 但是往往会发生不更新 modified_time 而导致的数据遗漏, 或者凌晨时间产生的数据记录漂移到后一天。

3、log_time 分区。由于网络或者系统压力问题, log_time 会晚于 proc_time, 从而导致凌晨时间产生的数据记录漂移到后一天。例如, 在淘宝“双 11”大促期间凌晨时间产生的数据量非常大, 用户支付需要调用多个接口, 从而导致 log_time 晚于实际的支付时间。根据 proc_time 业务过程分区。仅仅根据 proc_time 限制, 所获取的 ODS 表只是包含一个业务过程所产生的记录, 会遗漏很多其他过程的变化记录。

5.2.2 解决方案

(1) **多获取后一天的数据**: 既然很难解决数据漂移的问题, 那么就在 ODS 每个时间分区中向后多冗余一些数据, 在 ods 每个时间分区中向后多冗余一天数据, 保障数据只会多, 不会少, 而**具体的数据区分则可以让下游根据自身不同的业务场景用不同的业务时间 proc_time 来限制**。

但这种方式会有一些数据误差, 因为后一天的数据可能已经更新多次, 直接获取到的那条记录已经是更新多次后的状态了, 此时可以创建链表, 用时间来约束获取记录的状态。

(2) **多个时间戳字段限制时间**: 通过多个时间戳字段限制时间, 来获取**相对准确**的数据, 常见的操作步骤如下:

i、首先根据 log_time 分别冗余前一天最后 15 分钟的数据和后一天凌晨开始 15 分钟的数据, 并用 modified_time 过滤非当天数据, 确保数据不会因为系统问题而遗漏。

ii、然后根据 log_time 获取后一天 15 分钟的数据, 针对此数据, 按照主键根据 log_time 做升序排列去重。因为我们需要获取的是最接近当天记录变化的数据 (数据库日志将保留所有变化的数据, 但是落地到 ODS 表的是根据主键去重获取最后状态变化的数据)。

iii、最后将前两步的结果数据做**全外连接**, 通过限制业务时间 proc_time 来获取我们所需要的数据。

下面来看处理淘宝交易订单的数据漂移的实际案例。在处理交易订单时发现，有大批在 11 月 11 日 23:59:59 左右支付的交易订单漂移到了 11 月 12 日。主要原因是用户下单支付后系统需要调用支付宝的接口而有所延迟，从而导致这些订单最终生成的时间跨天了。即 modified_time 和 log_time 都晚于 proc_time。如果订单只有一个支付业务过程，则可以用支付时间来限制就能获取到正确的数据。但是往往实际订单有多个业务过程：下单、支付、成功，每个业务过程都有相应的时间戳字段，并不只有支付数据会漂移。因此，可以根据实际情况获取后 15 分钟的数据，并限制各个业务过程的时间戳字段（下单、支付、成功）都是“双 11”当天的，然后对这些数据按照订单的 **modified time 升序排列，获取每个订单首次数据变更的那条记录**。此外，还可以根据 log_time 分别冗余前一天最后 15 分钟的数据和当天凌晨开始 15 分钟的数据，并用 modified time 过滤非当天数据，针对每个订单按照 **log time 进行降序排列，取每个订单当天最后一次数据变更的那条记录**。最后将两份数据根据订单做全外连接，将漂移数据回补到当天数据中。

总而言之，在生产中数据漂移问题是存在的，只能通过一些规则限制获取**相对准确的数据**。比如约定通过修改时间来分区、这个字段如果存在不更新的情况就需要业务系统治理 3002

5.3 数据倾斜☆☆

Hadoop 能够进行对海量数据进行批处理的核心，在于它的分布式思想，通过多台服务器（节点）组成集群，共同完成任务，进行分布式的数据处理。

理想状态下，一个任务是由集群下所有机器共同承担执行任务，每个节点承担的任务应该相近，但实际上在并行处理过程中，分配到每台节点的数据量并不是均匀的，当大量的数据分配到某一个节点时（假设 10 个节点，5 亿数据），那么原本只需要 1 小时完成的工作，变成了其中 9 个节点不到 1 小时就完成了工作，而分配到了大量数据的节点，花了 5 个小时才完成

从最终结果来看，就是这个处理 10 亿数据的任务，集群花了 5 个小时才最终得出结果。大量的数据集中到了一台或者几台机器上计算，这些数据的计算速度远远低于平均计算速度，导致整个计算过程过慢，这种情况就是发生了数据倾斜。

5.3.1 原因分析

1)、**key 分布不均匀**，某些 key 的数量过于集中，存在大量相同值的数据例如存在大量异常值或空值。

2)、**业务数据本身的特性**，例如某个分公司或某个城市订单量大幅提升几十倍甚至几

百倍，对该城市的订单统计聚合时，容易发生数据倾斜。

3)、建表时考虑不周。

4)、某些 SQL 语句本身就有数据倾斜，两个表中关联字段存在大量空值，或是关联字段的数据不统一，例如在 A 表中值是大写，B 表中值是小写等情况。发生数据倾斜的 SQL 操作如下表所示：

关键词	情况	后果
Join	其中一个表较小，但是 key 集中。	分发到某一个或几个 Reduce/Stage 上的数据远高于平均值。
	大表与大表，但是分桶的判断字段 0 值或空值过多。	这些空值都由一个 Reduce/Stage 处理非常慢。
group by	group by 维度过小 某值的数量过多。	处理某值的 Reduce/Stage 非常耗时。
Count Distinct	某特殊值过多。	处理此特殊值的 Reduce/Stage 非常耗时

5.3.2 数据倾斜现象

MapReduce 任务：主要表现在 reduce 阶段卡在 99.99%，一直 99.99%不能结束。有一个多几个 reduce 卡住，各种 container 报错 OOM，读写的数据量极大，至少远远超过其它正常的 reduce 伴随着数据倾斜，会出现任务被 kill 等各种诡异的表现。

Spark 任务：绝大多数 task 执行得都非常快，但个别 task 执行的极慢。单个 Executor 执行时间特别久，整体任务卡在某个 stage 不能结束。Executor lost, OOM, Shuffle 过程出错，正常运行的任务突然失败。一直会有 executor 出现 OOM 的错误，但是其余的 executor 内存使用率却很低。

5.3.3 数据倾斜的危害

5.3.3.1 任务长时间挂起，资源利用率下降

在分阶段执行任务的场景下，阶段与阶段之间通常存在数据上的依赖关系，后一阶段任务需要等前一阶段执行完成才能开始，举个例子 Stage1 在 Stage0 之后执行，假如

Stage1 依赖 Stage0 产生的数据结果，那么 Stage1 必须等待 Stage0 执行完成后才能开始，如果这时 Stage0 因为数据倾斜问题，导致任务执行时长过长，或者直接挂起，那么 Stage1 将一直处于等待状态，整个作业也就一直挂起。这个时候，资源被这个作业占据，但是却只有极少数 task 在执行，造成计算资源的严重浪费，利用率下降

5.3.3.2 引发内存溢出，导致任务失败

数据发生倾斜时，可能导致大量数据集中在少数几个节点上，在计算执行中由于要处理的数据超出了单个节点的能力范围，最终导致内存被撑爆，报 OOM 异常，直接导致任务失败。

5.3.3.3 作业执行时间超出预期，导致后续依赖数据结果的作业出错

有时候作业与作业之间，并没有构建强依赖关系，而是通过执行时间的前后时间差来调度，当前置作业未在预期时间范围内完成执行，那么当后续作业启动时便无法读取到其所需要的最新数据，从而导致连续出错

5.3.4 处理方案

5.3.4.1 参数优化

通用优化：提高 shuffle 并行度，Spark 的 shuffle 并行度默认值是 200，建议根据服务器的情况进行调整。一般是集群 cpu 总和的 2-3 倍。当发生数据倾斜的时候，适当增大并行度，可以将原本被分配到同一个 Task 的不同 Key 分配到不同 Task。**缺陷：**它没有从根本上改变数据倾斜的本质和问题，只是说尽可能地去缓解和减轻。

map 端聚合，启动负载均衡，适用范围：group by 造成的数据倾斜，set `hive.map.aggr=true`；在 map 中会做部分聚集操作，效率更高但需要更多的内存 set `hive.groupby.skewindata=true`；默认 false，数据倾斜时负载均衡。当选项设定为 true，它使计算变成了两个 mapreduce，先在第一个中在 shuffle 过程 partition 时**随机**给 key 打标记，使每个 key 随机均匀分布到各个 reduce 上计算，但是这样只能完成部分计算，因为相同 key 没有分配到相同 reduce 上，所以需要第二次的 mapreduce，这次就回归正常 shuffle，但是数据分布不均匀的问题在第一次 mapreduce 已经有了很大的改善。

Reduce join 改为 Map join，适用范围：小表 join 大表，set

`hive.auto.convert.join = true`; hive 是否自动根据文件大小, 选择将 common join 转成 map join。set `hive.mapjoin.smalltable.filesize = 25000000`; 大表小表判断的阈值, 如果表的大小小于该值 25Mb, 则会被判定为小表。则会被加载到内存中运行, 将 commonjoin 转化成 mapjoin。一般这个值也就最多几百兆的样子。原理: mapjoin 优化就是在 Map 阶段完成 join 工作, 而不是像通常的 common join 在 Reduce 阶段按照 join 的列值进行分发数据到每个 Reduce 上进行 join 工作。这样避免了 Shuffle 阶段, 从而避免了数据倾斜。

注意: 这个操作会将所有的小表全量复制到每个 map 任务节点, 然后再将小表缓存在每个 map 节点的内存里与大表进行 join 工作。小表的大小的不能太大, 一般也就几百兆, 否则会出现 OOM 报错。

开启 Skewed Join, hadoop 中默认是使用 `hive.exec.reducers.bytes.per.reducer = 1000000000`, 也就是每个节点的 reduce 默认是处理 1G 大小的数据, 如果你的 join 操作也产生了数据倾斜, 那么可以在 hive 中设定 `set hive.optimize.skewjoin = true;set hive.skewjoin.key = skew_key_threshold` (default = 100000) 建议每次运行比较复杂的 sql 之前都可以设一下这个参数. 如果不知道设置多少, 可以就按官方默认的 1 个 reduce 只处理 1G 的算法, 那么 `skew_key_threshold = 1G/平均行长`. 或者默认直接设成 2500000000 (差不多算平均行长 4 个字节), **当开启 Skew Join 之后, 在运行时, 会对数据进行扫描并检测哪个 key 会出现倾斜, 对于会倾斜的 key, 用 map join 做处理, 不倾斜的 key 正常处理。即自适应数据倾斜纠正解决方案**

5.3.4.2 SQL 优化

1、count (distinct) 优化, 慎用 distinct, 先用 group 去重, 再 count 子查询。

```
--优化前
select count(distinct a) from test ;

--优化后
select count(x.a)
from
(select a from test group by a ) x
```

比如如下案例: UV 就是独立访客, 经常求的指标, 因为要 count distinct, 所以非常容易倾斜。例如: 一个表 tb 有两个字段, app_id, user_id, app_id 代表了不同的 app, 求每个 app 的 UV。普通的 SQL:

```
select
app_id,
```

```
count(distinct user_id)
from tb
group by app_id;
```

根据 **group by** 的执行原理，**app_id** 相同的会发到同一个 **reduce** 中处理。如果某一个 **app_id** 数据量很大（比如抖音远大于西瓜视频），那么就会产生倾斜。

因此，可以分为两个阶段，一个阶段通过 **group by** 对 **app_id** 和 **user_id** 进行去重，第二个阶段再计算 **UV**。

```
select
  app_id,
  sum(uv) as uv
from (
  select
    app_id,
    user_id,
    1 as uv
  from tb
  group by app_id, user_id
) a
group by app_id;
```

2、聚合类操作，发生数据倾斜，解决方法：阶段拆分-两阶段聚合，在需要聚合的 **key** 前加一个随机数的前后缀，这样就能得到非常均匀的 **key**，然后按这个加工之后的 **key** 进行第一次聚合之后，再对聚合的结果，按照原始 **key** 进行二次聚合，这样基本就不可能出现数据倾斜了。比如有实际场景如下：**PV** 和 **UV** 类似，同样需要 **group by** 之后聚合，但是不需要去重，具有可加性，因此可以分散计算，再相加。例如，对于 **user_id** 为 1, 2, 2, 3，计算 **PV** 时可以 1, 2 计算一次，结果是 2，然后 2, 3 计算一次，结果是 2，最后 **PV** = 2 + 2 = 4，结果正确。而 **UV** 的结果是 3，不能分散计算。

普通的 SQL：

```
select
  app_id,
  count(user_id)
from tb
group by app_id;
```

根据 **group by** 的执行原理，**app_id** 相同的会发到同一个 **reduce** 中处理。如果某一个 **app_id** 数据量很大（比如抖音远大于西瓜视频），那么就会产生倾斜。因此，可以分为

两个阶段，一个阶段加随机数把 app_id 分散聚合一次，第二个阶段再聚合。

```
select
  substring_index(tmp_app_id,"_",1) app_id,
  sum(pv) as pv
from (
  select
    concat(app_id, "_", round(rand()*10)) tmp_app_id,
    count(user_id) as pv
  from tb
  group by concat(app_id,"_",round(rand()*10))
) a
group by substring_index(tmp_app_id,"_",1);
```

3、大表 join 大表发生数据倾斜，大表拆分，倾斜部分单独处理，中间表分桶排序后 join。 举例：假设有两张表 table_a 表是一张近 3 个月买家交易明细表，字段如下，数据量较大。

Buyer_id (买家 id)	Seller_id (卖家 id)	order_num(订单数)
1113839839	1001	5

table_b 表是一张所有卖家信息表，数据量也很大，不满足 mapjoin 的条件。

Seller_id (卖家 id)	Seller_level (卖家评级)
1001	A

```
SELECT
  t1.buyer_id
  ,sum(case when t2.seller_level = 'A' then order_num end) as a_num
  ,sum(case when t2.seller_level = 'B' then order_num end) as b_num
  ,sum(case when t2.seller_level = 'C' then order_num end) as c_num
FROM table_a t1 --买家交易明细表
INNER JOIN table_b t2 --卖家信息表
ON t1.seller_id = t2.seller_id
GROUP BY t1.buyer_id;
```

但因为二八法则，某些卖家会拥有成千上万的买家，而大部分卖家的买家数量是较少的，两个表关联会发生数据倾斜。

优化方案 1 (降数量级)：转为 map join 避免 shuffle，table_b 由于数据量较大，无法直接 map join，所以考虑通过过滤 3 个月内没有发生交易的卖家，来减少 table_b 的数据量，使其达到满足 map join 的情况。

```

SELECT
    t1.buyer_id
    ,sum(case when t4.seller_level = 'A' then order_num end) as a_num
    ,sum(case when t4.seller_level = 'B' then order_num end) as b_num
    ,sum(case when t4.seller_level = 'C' then order_num end) as c_num
FROM table_a t1
INNER JOIN
    (
        SELECT
            t2.seller_id
            ,t2.seller_level
        FROM
            table_b t2
        INNER JOIN
            (
                SELECT
                    seller_id
                FROM table_a
                GROUP BY seller_id
            ) t3
        ON t2.seller_id = t3.seller_id
    ) t4 --过滤掉 table_b 中没有出现在 table_a 中的 seller_id
ON t1.seller_id = t4.seller_id
GROUP BY t1.buyer_id;

```

此方案在有些时候能够起作用，但大部分情况并不能解决问题，比如 table_b 经过过滤后，数量依然很大，还是不满足 map join 的条件。

优化方案 2 (拆表)：拆表一分为二，倾斜部分单独处理，首先创建一个临时表 temp_b，存放大卖家（买家超过 1 万个）的名单，然后 table_a 分别 join temp_b 和 table_b，结果 union all 起来即可。**此方案的适用情况最多，且最有效。**

```

--临时表存放 90 天内买家超过 10000 的大卖家
INSERT OVERWRITE TABLE temp_b
SELECT
    t1.seller_id
    ,t2.seller_level

```

```

FROM
(
SELECT
    seller_id
FROM table_a
GROUP BY seller_id
) t1
LEFT JOIN
(
SELECT
    seller_id
    ,seller_level
FROM table_b
) t2
ON t1.seller_id = t2.seller_id
AND count(buyer_id) > 10000;

--获取最终结果
SELECT
    t7.buyer_id
    ,sum(case when t7.seller_level = 'A' then t7.order_num end) as a_num
    ,sum(case when t7.seller_level = 'B' then t7.order_num end) as b_num
    ,sum(case when t7.seller_level = 'C' then t7.order_num end) as c_num
FROM
(
SELECT
    /*+mapjoin(t2)*/
    t1.buyer_id
    ,t1.seller_id
    ,t1.order_num
    ,t2.seller_level
FROM table_a t1
LEFT JOIN temp_b t2
ON t1.seller_id = t2.seller_id

```


UNION ALL --针对大卖家 map join 其他卖家正常 join

```
SELECT
    t3.buyer_id
    ,t3.seller_id
    ,t3.order_num
    ,t6.seller_level
FROM
    table_a t3
LEFT JOIN
    (
        SELECT
            seller_id
            ,seller_level
        FROM table_b t4
        LEFT JOIN temp_b t5
        ON t4.seller_id = t5.seller_id
        WHERE t5.seller_id is null
    ) t6
ON t3.seller_id = t6.seller_id
) t7
GROUP BY t7.buyer_id;
```

4、空值处理

例如埋点日志经常丢失 user_id，存在大量空值。所以空值到一个 reduce 中发生倾斜。

解决方法：空值赋新的 key 值。

```
select *
from logs a left join users b
on
case
    when a.user_id is null then concat( 'hive' ,rand() )
    else a.user_id
end
= b.user_id;
```

总结：

join 发生数据倾斜通用方法，解决方法： 随机前缀和 RDD 扩容，sql 实现方法：

- 1、将大表的倾斜 key 筛选出来，随机打上前后缀，假如为 1-10 的数字。
- 2、将较小表 cross join(笛卡尔积) 1-10 数字共 10 行，再通过拼接，形成与大表对应的前后缀。(假如较小表数据量为 10 万，此时扩容至 100 万)。
- 3、此时将筛选过的大表与扩容的较小表关联，关联成功后，通过字段加工复原。
- 4、将大表其他无倾斜数据与较小表关联，union all 前 3 步处理后得到的数据。

空值产生的数据倾斜，解决方法：给空值进行随机赋值，场景：如日志中，常会有信息丢失的问题，比如日志中的 user_id，如果取其中的 user_id 和用户表中的 user_id 关联，会碰到数据倾斜的问题 3002

5.3.4.3 自定义 partitioner (对代码有要求)

根据现在处理的这份数据，单独写一个适合的 partitioner。比如现在是按省份进行汇总数据，如果只是简单的按省份进行聚合，那么数据肯定会倾斜，因为各省的数据天然不一样。我们可以通过历史数据、抽样数据或者一些常识，对数据进行人工分区，让数据按照我们自定义的分区规则比较均匀的分配到不同的 task 中。

常见的分区方式：

随机分区：每个区域的数据基本均衡，简单易用，偶尔出现倾斜，但是特征同样也会随机打散。

轮询分区：绝对不会倾斜，但是需要提前预知分成若干份，进行轮询。

hash 散列：可以针对某个特征进行 hash 散列，保证相同特征的数据在一个区，但是极容易出现数据倾斜。

范围分区：需要排序，临近的数据会被分在同一个区，可以控制分区数据均匀 3002